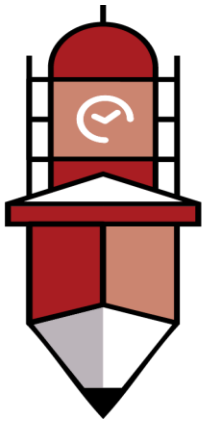
Data Mining (DS-303)

LAB MANUAL



The University of Faisalabad

SUBMITTED BY	Abdullah Mehfooz
SUBMITTED TO	Sir Arham
REGISTRATION NO	2023-BS-AI-170

DEPARTMENT OF COMPUTATIONAL SCIENCES

FACULTY OF INFORMATION TECHNOLOGY

The University of Faisalabad

–

Table of Contents

–

Lab No. 08: Python Programming Fundamentals and Object-Oriented Programming

- **Tools Used**
  Python

Lab No. 09: Decision Tree Classification

- **Tools Used** KNIME
  Python

Lab No. 10: Naïve Bayes and K-Nearest Neighbors (KNN)

- **Tools Used** RapidMiner
  Python

Lab No. 11: Support Vector Machine (SVM)

- **Tools Used** KNIME
  Python

Lab No. 12: Clustering Techniques

- **Tools Used** Orange
  Data Mining
  Python

Lab No. 13: Outlier Detection and Analysis

- **Tools Used**
  RapidMiner
  Python

Lab No. 14: Web Scraping and Sentiment Analysis

- **Tools Used**
  CiteSpace
  Python

–

Lab No. 15: Final Capstone Project

- **Tools Used**

  KNIME Analytics Platform
  Orange Data Mining
  RapidMiner
  Python

–

Lab No. 01: Introduction to Data Analytics and KNIME Platform

Experiment Title

Introduction to Data Mining

Business Scenario

A retail supermarket organization intends to analyze its sales transactions to uncover valuable business insights. The objective is to evaluate sales performance, customer purchasing behavior, and product category trends for better decision-making.

Dataset

Supermarket Sales Dataset (Kaggle)

Software Used

KNIME Analytics Platform

Key Performance Indicators (KPIs)

- Total Sales Revenue
- Average Transaction Value
- Revenue by Product Category
- Top Performing Products

Objectives

- Import and inspect a real-world retail dataset.
- Identify and verify data types of dataset attributes.
- Generate descriptive statistics for important variables.
- Explore data patterns and attribute distributions.
- Develop basic visualizations using KNIME.

–

Introduction

Data Mining is the process of extracting meaningful patterns, relationships, trends, and useful information from large datasets. Organizations utilize data mining techniques to transform raw data into actionable knowledge that supports strategic and operational decisions.

In this laboratory exercise, students will work with a retail supermarket sales dataset to understand how transactional records can be analyzed to produce business intelligence.

KNIME Analytics Platform is a graphical data analytics tool that enables users to build analytical workflows through a drag-and-drop interface. It supports data preparation, transformation, analysis, and visualization without extensive programming knowledge.

Dataset Overview

Attribute	Data Type	Description
Invoice ID	String	Unique identifier assigned to each transaction
Branch	String	Store branch code (A, B, or C)
City	String	City where the transaction occurred
Customer Type	String	Membership status (Member or Normal)
Gender	String	Gender of the customer
Product Line	String	Category of purchased products
Unit Price	Float	Price per individual item
Quantity	Integer	Number of units purchased
Total	Float	Total transaction amount
Date	Date	Date of purchase
Payment	String	Payment method used
Rating	Float	Customer satisfaction rating (1–10)

Procedure

Step 1: Import the Dataset

1. Launch KNIME Analytics Platform.
2. Create a new workflow named **“Lab01_Data_Mining”**.
3. Drag and drop a **CSV Reader** node into the workflow.
4. Configure the node by selecting the Supermarket Sales dataset file.
5. Execute the node and review the imported data.
6. Verify that all columns have been loaded correctly.

Expected Outcome

–

The dataset is successfully imported and available for analysis.

Step 2: Examine Data Types

1. Connect a **Statistics** node to the CSV Reader.
2. Execute the node and inspect attribute information.
3. Use **Data Explorer** or **Column Filter** nodes to review column types.
4. Confirm that fields such as:
   - Unit Price
   - Quantity
   - Total
   - Rating

   are recognized as numeric variables.

5. Correct any incorrectly assigned data types if necessary.

Expected Outcome

All dataset attributes are assigned appropriate data types for analysis.

Step 3: Generate Statistical Summaries

1. Connect a **Statistics** node to analyze numerical attributes.
2. Review:
   - Mean
   - Minimum
   - Maximum
   - Standard Deviation
   - Count
3. Analyze the following fields:
   - Total
   - Quantity
   - Unit Price
   - Rating
4. Add a **GroupBy** node.
5. Group records using **Product Line**.
6. Calculate:

–

- Sum(Total)
- Count(Invoice ID)

7. Use a **Sorter** node to rank categories by revenue.

Expected Outcome

Summary statistics and category-level revenue analysis are generated successfully.

Step 4: Create Data Visualizations

1. Add a **Bar Chart** node from the visualization section.
2. Configure:
   - X-Axis: Product Line
   - Y-Axis: Sum(Total)
3. Execute the visualization.
4. Review the generated chart.
5. Export the chart using an **Image Writer** node.

Expected Outcome

A visual representation of category-wise revenue is produced and saved.

KPI Analysis

KPI	Formula	Business Purpose
Total Revenue	SUM(Total)	Measures overall sales performance
Average Transaction Value	SUM(Total) / COUNT(Invoice ID)	Evaluates customer spending behavior
Category Revenue	SUM(Total) GROUP BY Product Line	Identifies profitable product categories
Top 5 Products	Sort by Total (Descending)	Highlights high-value products

Expected Results

- Approximately 1,000 transaction records should be available.
- Food & Beverages or Electronic Accessories may emerge as the highest revenue-generating categories.
- Average transaction value is expected to range between $300 and $350.
- Most customer ratings are likely to fall between 5.5 and 8.5.

–

Assessment Tasks

Task 1

Import the dataset and determine the total number of rows and columns.

Task 2

Identify any missing or null values present in the dataset.

Task 3

Calculate total revenue generated by each product category.

Task 4

Determine the top five transactions based on total sales amount.

Task 5

Create a bar chart showing revenue by product category and include it in the lab report.

Task 6

Identify the branch (A, B, or C) with the highest average customer rating.

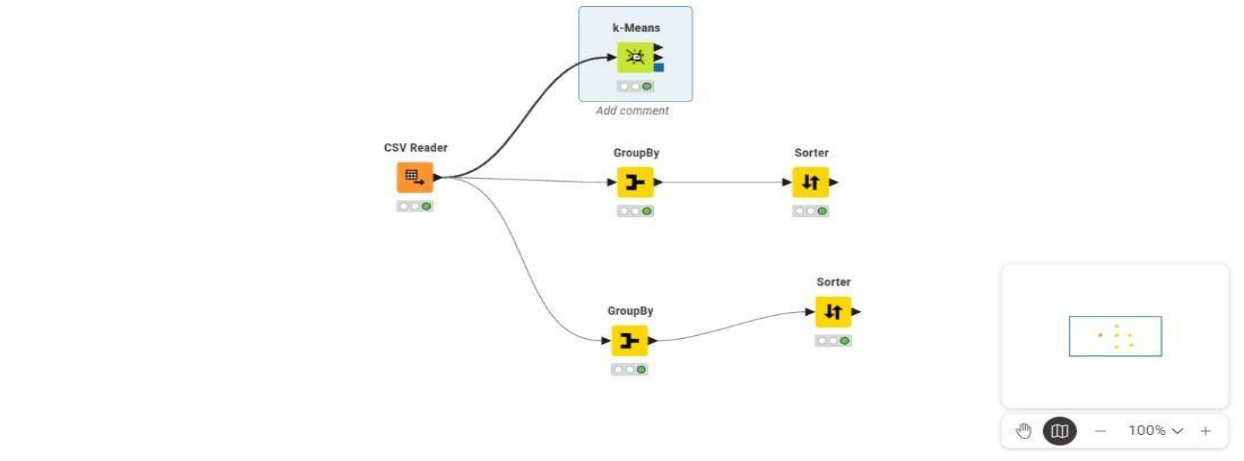
Conclusion

Upon completing this lab, students will gain practical experience in importing, exploring, summarizing, and visualizing business data using KNIME Analytics Platform. The exercise demonstrates how data mining techniques can be applied to transform raw transactional records into meaningful business insights.

–

Open for Innovation

KNIME



–

LAB 02: CRISP-DM Framework for Customer Churn Analysis

Topic

CRISP-DM Process Model

Business Scenario

An e-commerce company wants to identify customers who are likely to discontinue their services so that retention strategies can be implemented proactively.

Dataset

Customer Churn Dataset (Kaggle)

Software Used

Orange Data Mining

Key Performance Indicators (KPIs)

• Churn Rate

• Customer Retention Rate

• Prediction Accuracy • ROC-AUC Score **Objectives**

• Understand the complete CRISP-DM lifecycle and its applications.

• Apply the CRISP-DM methodology to a customer churn prediction problem.

• Develop predictive models using Orange Data Mining.

• Assess model performance using industry-standard evaluation metrics.

Introduction to CRISP-DM

CRISP-DM (Cross-Industry Standard Process for Data Mining) is a widely adopted framework that provides a structured approach to data mining projects. The methodology consists of six interconnected phases that guide analysts from problem identification to solution deployment. Its iterative nature allows continuous improvement throughout the project lifecycle.

CRISP-DM Phases

Phase	Purpose	Deliverables
Business	Define project objectives and business	

–

Understanding	requirements	Project goals, success criteria, KPIs
Data Understanding	Collect and explore available data	Data exploration findings, quality assessment
Data Preparation	Clean and transform data for analysis	Processed dataset ready for modeling
Modeling	Develop predictive or descriptive models	Trained machine learning models
Evaluation	Assess model effectiveness	Performance reports and validation results
Deployment	Implement the solution for practical use	Operational model and documentation

Phase 1: Business Understanding

Business Objective

The primary objective is to predict customer churn before it occurs. By identifying high-risk customers, the organization can provide targeted incentives and personalized retention campaigns.

Success Criteria

• Model Accuracy of at least 80%

• ROC-AUC Score of 0.75 or higher

Project Constraints

• Results should be interpretable and easy to explain to business stakeholders.

• The solution should support data-driven retention strategies.

Phase 2: Data Understanding

The Customer Churn dataset contains customer demographic and subscription-related information that can be used to identify churn patterns.

Dataset Attributes

• CustomerID – Unique customer identifier

• Tenure – Number of months the customer has remained subscribed

• MonthlyCharges – Monthly subscription fee
• TotalCharges – Total amount billed over the customer lifecycle

• Contract – Subscription type (Month-to-Month, One Year, Two Years)

• Churn – Target variable indicating whether the customer left the service

–

Phase 3: Data Preparation

Load Dataset in Orange

1. Open Orange Data Mining.
2. Drag a File widget onto the workspace.
3. Import the Customer Churn CSV dataset.
4. Connect the File widget to a Data Table widget.
5. Inspect records and attribute values.

Handle Missing Data

1. Add an Impute widget.
2. Configure missing-value handling:
   - Median replacement for numerical attributes
   - Most frequent value replacement for categorical attributes
3. Connect the output to a Data Info widget.
4. Verify that all missing values have been successfully handled.

Select Relevant Features

1. Add a Select Columns widget.
2. Remove CustomerID since it does not contribute to prediction.
3. Set Churn as the target (class) variable.
4. Retain all relevant predictive attributes.

Phase 4: Model Development

Build Classification Models

1. Add a Logistic Regression learner.
2. Add a Naïve Bayes learner.
3. Connect both learners to a Test and Score widget.
4. Configure 10-fold cross-validation for model assessment.
5. Execute the workflow and record performance metrics.

Phase 5: Model Evaluation

The effectiveness of the predictive models can be assessed using the following measures:

Metric	Calculation	Expected Goal
Accuracy	Correct Predictions / Total Predictions	≥ 80%
Churn Rate	Churned Customers / Total Customers	Identify at-risk customers
Retention Rate	1 − Churn Rate	Measure customer loyalty

–

ROC-AUC	Area Under the ROC Curve	≥ 0.75

Expected Outcomes

• A predictive model capable of identifying potential churners.

• Accurate classification of customer retention behavior.

• Insights into the factors most strongly associated with churn.

• Comparative performance analysis of multiple classification techniques.

Lab Exercises

Task 1

Relate the customer churn prediction problem to each of the six CRISP-DM phases. Provide a brief explanation for every phase.

Task 2

Import the dataset into Orange and calculate the overall churn rate.

Task 3

Apply missing-value treatment using the Impute widget and compare the number of missing values before and after processing.

Task 4

Train a Logistic Regression model and report its Accuracy and ROC-AUC values.

Task 5

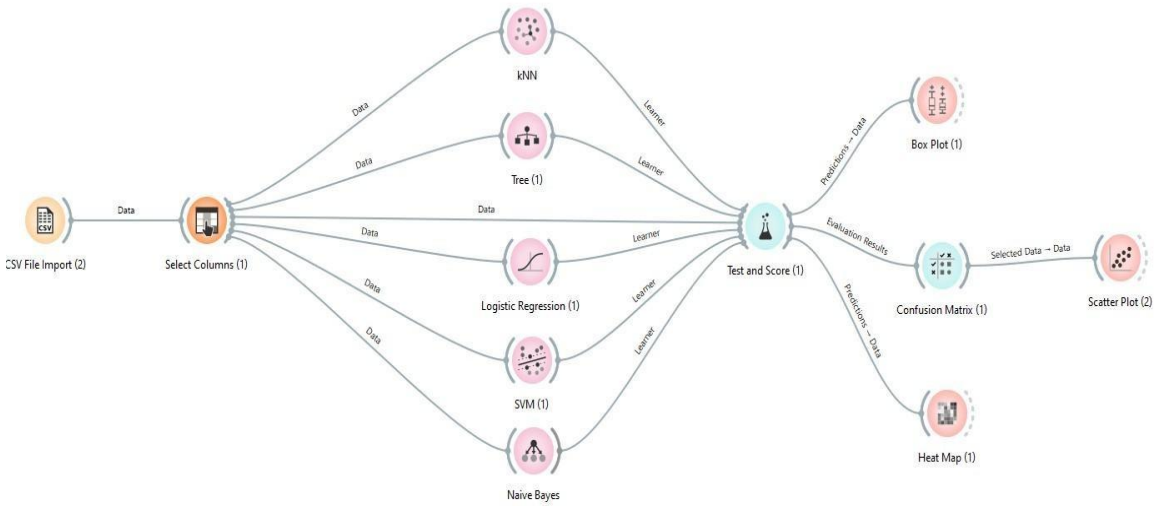
Compare Logistic Regression and Naïve Bayes models. Determine which model provides better predictive performance.

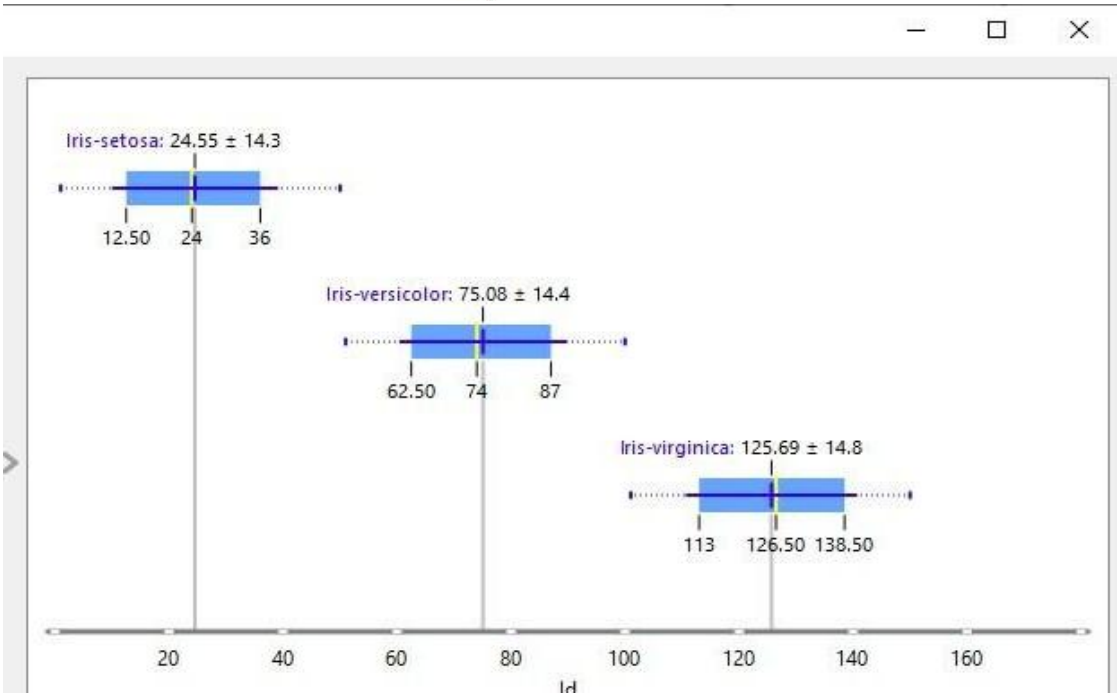
Task 6

Identify the feature that exhibits the strongest relationship with customer churn and justify your findings using data analysis results.

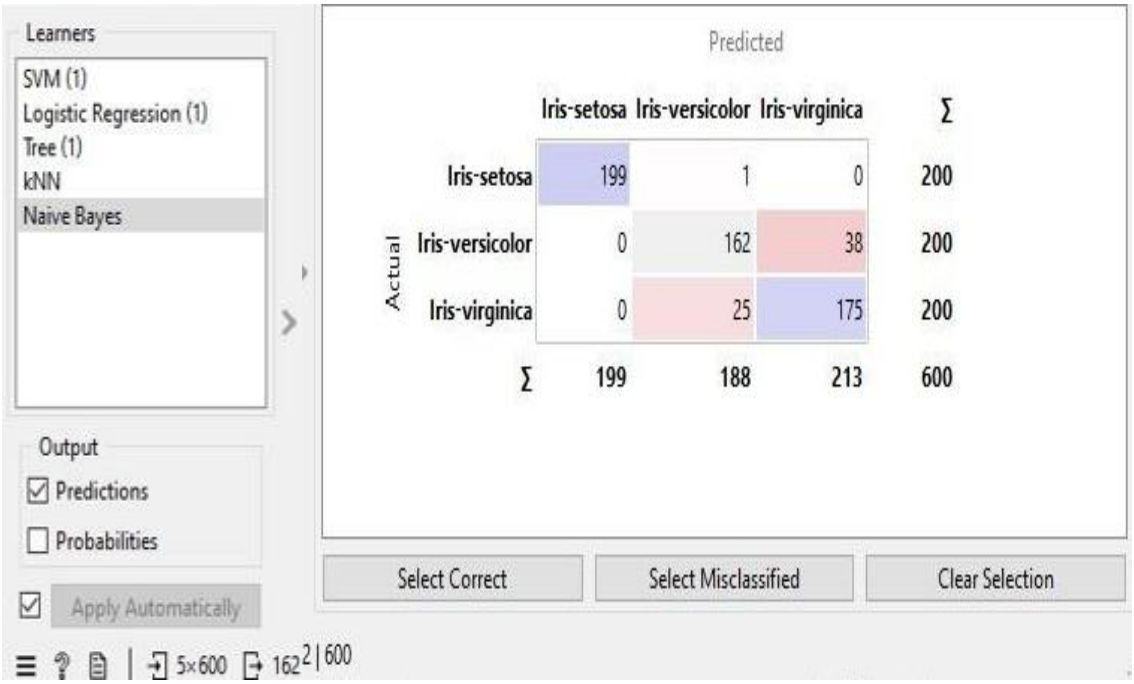
Conclusion

This laboratory exercise introduces the CRISP-DM methodology through a real-world customer churn prediction problem. Students gain practical experience in understanding business requirements, preparing data, building predictive models, evaluating outcomes, and applying a structured framework for data mining projects.

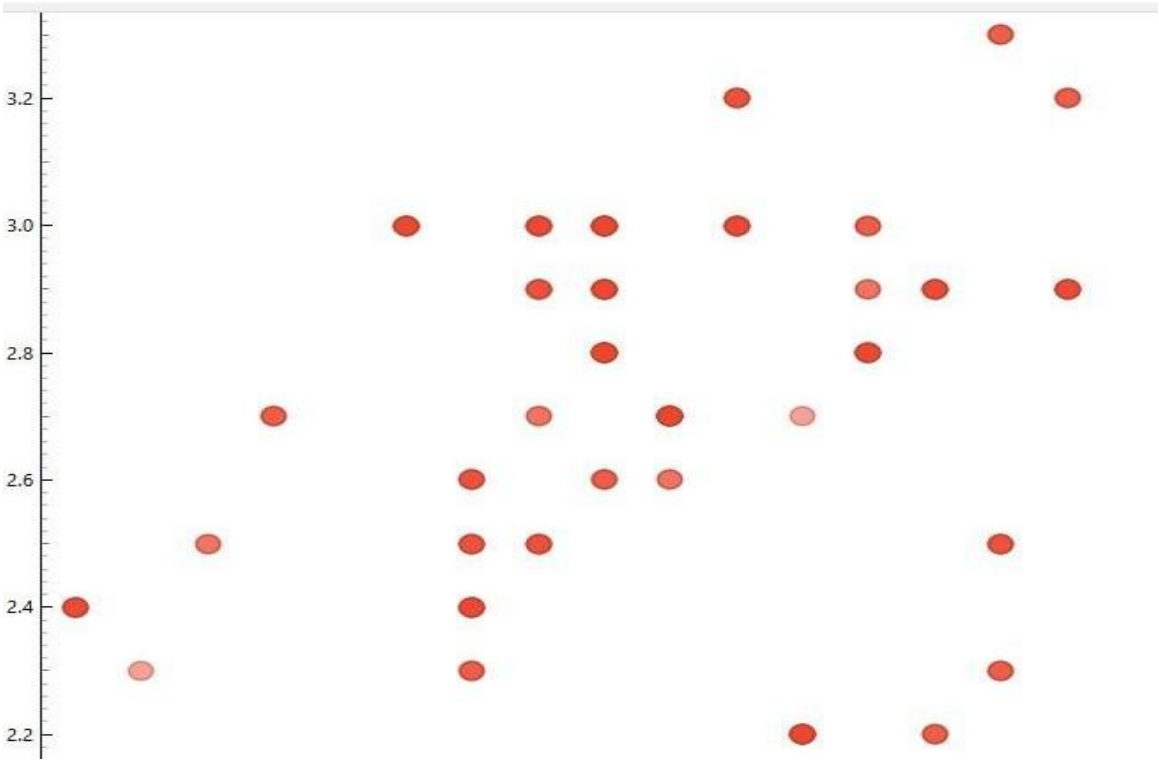

kNN
Tree (1)
Logistic Regression (1)
SVM (1)
Naive Bayes
CSV File Import (2)
Select Columns (1)
Test and Score (1)
Box Plot (1)
Confusion Matrix (1)
Heat Map (1)
Scatter Plot (2)
Data
Learner
Predictions → Data
Evaluation Results
Selected Data → Data


Iris-setosa: 24.55 ± 14.3
12.50 24 36
Iris-versicolor: 75.08 ± 14.4
62.50 74 87
Iris-virginica: 125.69 ± 14.8
113 126.50 138.50
20 40 60 80 100 120 140 160
Id

–


Learners
SVM (1)
Logistic Regression (1)
Tree (1)
kNN
Naive Bayes
Output
Predictions
Probabilities
Apply Automatically
Predicted
Iris-setosa Iris-versicolor Iris-virginica Σ
Actual
Iris-setosa 199 1 0 200
Iris-versicolor 0 162 38 200
Iris-virginica 0 25 175 200
Σ 199 188 213 600
Select Correct
Select Misclassified
Clear Selection
5×600
162 2 | 600

–


3.2
3.0
2.8
2.6
2.4
2.2

–

Lab No. 03: Data Cleaning Techniques

Topic

Data Cleaning and Quality Improvement

Business Scenario

A healthcare organization is preparing patient records for predictive analytics. Before developing machine learning models, the dataset must be cleaned to eliminate inaccuracies, missing values, duplicate entries, and abnormal observations.

Dataset

Synthetic Healthcare Dataset

Software Used

Python (pandas, numpy, matplotlib, scipy)

Key Performance Indicators (KPIs)

• Percentage of Missing Values Reduced

• Duplicate Record Removal Percentage

• Data Quality Score

Objectives

• Identify incomplete data and apply appropriate imputation techniques.

• Detect and eliminate duplicate records.

• Identify and treat noisy data and outliers using statistical methods.

• Measure and evaluate data quality through performance indicators.

• Prepare a clean dataset suitable for predictive modeling.

Introduction

Data quality plays a critical role in the success of data analysis and machine learning projects. Real-world healthcare datasets frequently contain incomplete records, duplicate entries, inconsistent information, and extreme values that can negatively impact analytical outcomes.

–

Data cleaning is the process of detecting and correcting such issues to improve the reliability, consistency, and accuracy of data. In healthcare applications, poor-quality data can lead to incorrect predictions and unreliable decision-making. Therefore, data preprocessing is an essential step before building predictive models.

This lab focuses on improving healthcare data quality through missing value treatment, duplicate removal, outlier detection, and KPI-based evaluation.

Dataset Description

The synthetic healthcare dataset contains patient-related information commonly used in predictive healthcare applications.

Sample Attributes

• Patient ID – Unique patient identifier

• Age – Patient age in years

• Gender – Patient gender

• Blood Pressure – Recorded blood pressure measurement

• Glucose Level – Blood glucose reading

• Cholesterol – Cholesterol measurement

• Diagnosis – Medical condition category

• Admission Type – Type of hospital admission

Data Quality Metrics

KPI	Calculation	Target
Missing Value Reduction (%)	(Missing Before − Missing After) / Missing Before × 100	≥ 90%
Duplicate Removal (%)	Duplicates Removed / Original Records × 100	100%
Data Quality Score (%)	(Valid Cells / Total Cells) × 100	≥ 95%

Data Cleaning Procedure

Step 1: Load and Inspect the Dataset

1. Import the required Python libraries:
    - Pandas
    - numpy

–

- matplotlib
- scipy

2. Load the healthcare dataset into a DataFrame.
3. Examine: ○ Dataset dimensions ○ Data types ○ Missing values ○ Duplicate records
4. Generate an initial data quality assessment.

Expected Outcome

The dataset is successfully loaded and potential quality issues are identified.

Step 2: Missing Value Analysis

1. Calculate the number of missing values in each column.
2. Visualize missing data patterns using a heatmap.
3. Identify attributes with the highest proportion of missing values.
4. Assess the impact of missing data on analysis.

Expected Outcome

Missing value distribution is clearly identified and documented.

Step 3: Missing Value Treatment

1. Apply median imputation to numerical attributes.
2. Apply mode imputation to categorical attributes.
3. Recalculate missing value counts after imputation.
4. Verify that all missing values have been handled appropriately.

Expected Outcome

The dataset contains minimal or no missing values after treatment.

Step 4: Duplicate Record Detection

1. Identify duplicate rows within the dataset.
2. Calculate the total number of duplicate records.
3. Remove duplicate entries.
4. Compare dataset size before and after duplicate removal.

Expected Outcome

All duplicate records are successfully eliminated.

–

Step 5: Outlier Detection and Treatment

1. Identify unrealistic values using domain knowledge.
2. Detect statistical outliers using Z-score analysis.
3. Review abnormal observations such as: ○ Extremely high age values ○ Unusual blood pressure readings ○ Abnormal glucose levels
4. Remove or correct identified outliers.

Expected Outcome

The dataset contains only valid and realistic observations.

Step 6: Calculate Data Quality KPIs

After cleaning the dataset, calculate:

• Missing Value Reduction Percentage

• Duplicate Removal Percentage

• Data Quality Score

Compare the results against the predefined targets.

Expected Outcome

Data quality metrics meet or exceed the established thresholds.

Expected Results

• Significant reduction in missing values. • Complete removal of duplicate records.

• Successful identification and treatment of outliers.

• Data Quality Score exceeding 95%.

• A clean dataset ready for predictive modeling.

Lab Exercises

Task 1

Execute the synthetic dataset generation process and report the dataset dimensions before and after cleaning.

Task 2

Create a missing-value heatmap and determine which attribute contains the highest number of missing values.

Task 3

After applying imputation techniques, verify that no missing values remain. Display the output of:

df_clean.isnull().sum()

Task 4

Identify the outlier values intentionally introduced into the Age, Blood Pressure, and Glucose attributes. Explain whether these values were removed or corrected.

Task 5

Calculate and report the following KPIs:

• Missing Value Reduction Percentage

• Duplicate Removal Percentage

• Data Quality Score **Task 6**

Compare the dataset quality before and after cleaning and discuss the impact of preprocessing on data reliability.

Conclusion

This laboratory exercise demonstrates the importance of data cleaning in healthcare analytics. Through missing value treatment, duplicate removal, and outlier detection, students learn how to improve dataset quality and prepare reliable data for predictive modeling and advanced analytical tasks.

```
[1]
✓ 1s
        import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        import seaborn as sns
        from scipy import stats

        print('Libraries loaded successfully!')

        Libraries loaded successfully!
```

```
np.random.seed(42)
n = 300

df = pd.DataFrame({
    'PatientID':     range(1, n+1),
    'Age':           np.random.randint(18, 90, n).astype(float),
    'Gender':        np.random.choice(['Male','Female', None], n, p=[0.48,0.48,0.04]),
    'BloodPressure': np.random.normal(120, 15, n),
    'Cholesterol':   np.random.normal(200, 40, n),
    'Glucose':       np.random.normal(100, 25, n),
    'Diagnosis':     np.random.choice(['Healthy','Diabetic','Hypertensive','Cardiac', None], n, p=[0.4,0.2,0.2,0.15,0.05])
})

# Inject missing values
for col in ['Age','BloodPressure','Cholesterol','Glucose']:
    idx = np.random.choice(df.index, size=int(0.08*n), replace=False)
    df.loc[idx, col] = np.nan

# Inject duplicates
df = pd.concat([df, df.sample(15, random_state=1)], ignore_index=True)

# Inject noise / outliers
df.loc[5,  'Age']          = 180
df.loc[10, 'BloodPressure'] = 400
df.loc[15, 'Glucose']      = -50

print('Dataset Shape:', df.shape)
df.head(10)
```

```
Dataset Shape: (315, 7)
```

	PatientID	Age	Gender	BloodPressure	Cholesterol	Glucose	Diagnosis
0	1	69.0	Female	125.402535	244.121011	177.621686	Healthy
1	2	32.0	Male	145.778991	NaN	NaN	Healthy

–

```
print('=== Dataset Info ===')
df.info()
print()
print('=== Summary Statistics ===')
df.describe()
```

```
=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 315 entries, 0 to 314
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   PatientID      315 non-null    int64
 1   Age            290 non-null    float64
 2   Gender         302 non-null    object
 3   BloodPressure  291 non-null    float64
 4   Cholesterol    289 non-null    float64
 5   Glucose        288 non-null    float64
 6   Diagnosis      299 non-null    object
dtypes: float64(4), int64(1), object(2)
memory usage: 17.4+ KB

=== Summary Statistics ===
```

```
missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100
total_missing_before = missing.sum()

print('=== Missing Values Before Cleaning ===')
print(pd.DataFrame({'Count': missing, 'Percentage (%)': missing_pct.round(2)}))
print(f'\nTotal Missing Cells: {total_missing_before}')

# Heatmap
plt.figure(figsize=(10, 4))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False)
plt.title('Missing Value Map (Yellow = Missing)', fontsize=14)
plt.tight_layout()
plt.show()
```

```
=== Missing Values Before Cleaning ===
               Count  Percentage (%)
PatientID          0            0.00
Age               25            7.94
Gender            13            4.13
BloodPressure     24            7.62
Cholesterol       26            8.25
Glucose           27            8.57
Diagnosis         16            5.08

Total Missing Cells: 131
```

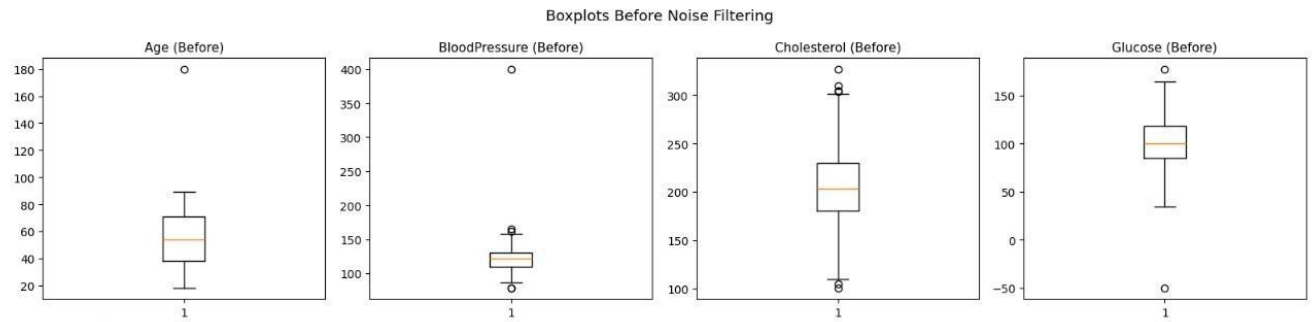
```
df_clean = df.copy()

num_cols = ['Age', 'BloodPressure', 'Cholesterol', 'Glucose']
for col in num_cols:
    median_val = df_clean[col].median()
    df_clean[col].fillna(median_val, inplace=True)
    print(f'{col}: filled missing with median = {median_val:.2f}')

for col in ['Gender', 'Diagnosis']:
    mode_val = df_clean[col].mode()[0]
    df_clean[col].fillna(mode_val, inplace=True)
    print(f'{col}: filled missing with mode = {mode_val}')

print(f'\nMissing After Imputation: {df_clean.isnull().sum().sum()}')
```

```
Age: filled missing with median = 54.00
BloodPressure: filled missing with median = 121.14
Cholesterol: filled missing with median = 203.19
Glucose: filled missing with median = 100.80
Gender: filled missing with mode = Male
Diagnosis: filled missing with mode = Healthy

Missing After Imputation: 0
/tmp/ipykernel_2733/2036428332.py:6: FutureWarning: A value is trying
The behavior will change in pandas 3.0. This inplace method will never

For example, when doing 'df[col].method(value, inplace=True)', try usi


  df_clean[col].fillna(median_val, inplace=True)
/tmp/ipykernel_2733/2036428332.py:11: FutureWarning: A value is trying
The behavior will change in pandas 3.0. This inplace method will never

For example, when doing 'df[col].method(value, inplace=True)', try usi


  df_clean[col].fillna(mode_val, inplace=True)
```

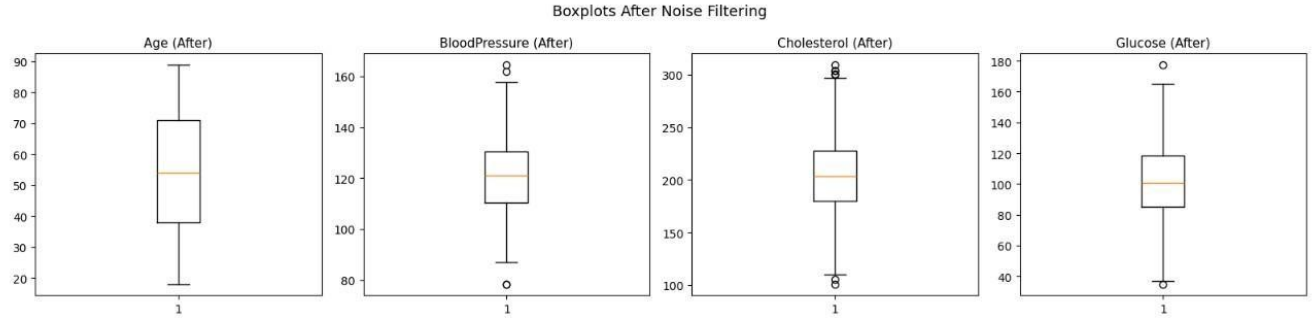
```
duplicates_before = df_clean.duplicated().sum()
print(f'Duplicates Found: {duplicates_before}')
df_clean = df_clean.drop_duplicates().reset_index(drop=True)
dup_removal_pct = (duplicates_before / len(df)) * 100
print(f'Rows Before: {len(df)}  |  Rows After: {len(df_clean)}')
print(f'KPI - Duplicate Removal %: {dup_removal_pct:.2f}%')
```

```
Duplicates Found: 15
Rows Before: 315  |  Rows After: 300
KPI - Duplicate Removal %: 4.76%
```

```python
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for i, col in enumerate(num_cols):
    axes[i].boxplot(df_clean[col].dropna())
    axes[i].set_title(f'{col} (Before)', fontsize=11)
plt.suptitle('Boxplots Before Noise Filtering', fontsize=13)
plt.tight_layout()
plt.show()

# Domain-Based Rules
domain_rules = {'Age':(0,120), 'BloodPressure':(50,250), 'Cholesterol':(50,500), 'Glucose':(30,500)}
rows_before = len(df_clean)
for col, (lo, hi) in domain_rules.items():
    outliers = ((df_clean[col] < lo) | (df_clean[col] > hi)).sum()
    df_clean = df_clean[(df_clean[col] >= lo) & (df_clean[col] <= hi)]
    print(f'{col}: removed {outliers} domain-rule outliers')

# Z-Score Filter
z_scores = np.abs(stats.zscore(df_clean[num_cols]))
df_clean = df_clean[(z_scores < 3).all(axis=1)].reset_index(drop=True)
print(f'\nFinal Clean Dataset Shape: {df_clean.shape}')

# After boxplots
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for i, col in enumerate(num_cols):
    axes[i].boxplot(df_clean[col].dropna())
    axes[i].set_title(f'{col} (After)', fontsize=11)
plt.suptitle('Boxplots After Noise Filtering', fontsize=13)
plt.tight_layout()
plt.show()
```




```
Age: removed 1 domain-rule outliers
BloodPressure: removed 1 domain-rule outliers
Cholesterol: removed 0 domain-rule outliers
Glucose: removed 1 domain-rule outliers

Final Clean Dataset Shape: (296, 7)
```



```
total_cells_original = df.shape[0] * df.shape[1]
total_missing_after  = df_clean.isnull().sum().sum()
missing_reduced_pct  = ((total_missing_before - total_missing_after) / total_missing_before * 100) if total_missing_before > 0 else 100
data_quality_score   = ((total_cells_original - total_missing_before) / total_cells_original) * 100

print('=' * 45)
print('       LAB 3 – DATA QUALITY KPI REPORT')
print('=' * 45)
print(f'Original Rows         : {len(df)}')
print(f'Clean Rows            : {len(df_clean)}')
print(f'% Missing Reduced     : {missing_reduced_pct:.1f}%')
print(f'Duplicate Removal %   : {dup_removal_pct:.2f}%')
print(f'Data Quality Score    : {data_quality_score:.1f}%')
print('=' * 45)

plt.figure(figsize=(8, 5))
kpi_labels = ['% Missing\nReduced', 'Duplicate\nRemoval %', 'Data Quality\nScore']
kpi_values = [missing_reduced_pct, dup_removal_pct, data_quality_score]
colors     = ['#4CAF50', '#2196F3', '#FF9800']
bars = plt.bar(kpi_labels, kpi_values, color=colors, edgecolor='black', width=0.5)
for bar, val in zip(bars, kpi_values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1, f'{val:.1f}%', ha='center', fontsize=12, fontweight='bold')
plt.ylim(0, 115)
plt.ylabel('Percentage (%)', fontsize=12)
plt.title('Lab 3 – Data Cleaning KPI Results', fontsize=14)
plt.tight_layout()
plt.show()

df_clean.to_csv('healthcare_cleaned.csv', index=False)
print('Saved: healthcare_cleaned.csv')
```


28

–

Lab No. 04: Data Preprocessing and Transformation

Topic

Normalization, Standardization, and Discretization

Business Scenario

A banking institution is preparing loan application data for machine learning models. Before model development, the dataset must be transformed into a suitable format through scaling, encoding, and feature engineering techniques.

Dataset

Synthetic Loan Prediction Dataset

Software Used

Python (pandas, scikit-learn, matplotlib)

Key Performance Indicators (KPIs)

• Variance Reduction

• Distribution Improvement

Objectives

• Apply Min-Max Scaling to transform numerical features into a common range.

• Perform Z-Score Standardization to create features with zero mean and unit variance.

• Convert continuous variables into categorical intervals through discretization.

• Encode categorical attributes into numerical values using Label Encoding.

• Analyze and compare data distributions before and after preprocessing.

Introduction

Data preprocessing is a crucial stage in the machine learning pipeline. Raw data often contains features with different scales, formats, and distributions that may negatively affect model

performance. Proper preprocessing improves data consistency, enhances model accuracy, and facilitates efficient learning.

Common preprocessing techniques include normalization, standardization, discretization, and categorical encoding. These methods help prepare data for algorithms such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Decision Trees, and Neural Networks.

In this lab, students will apply various preprocessing methods to a loan prediction dataset and evaluate their impact on data quality and feature distribution.

Dataset Description

The synthetic loan dataset contains customer and financial information commonly used for loan approval and risk assessment.

Sample Attributes

• Applicant ID – Unique application identifier

• Age – Age of the applicant

• Income – Monthly or annual income

• Loan Amount – Requested loan value

• Credit Score – Customer credit rating

• Employment Status – Employment category

• Education Level – Educational qualification • Loan Status – Approved or Rejected

Preprocessing Techniques

Technique	Formula / Method	Purpose
Min-Max Normalization	$x' = (x - \min) / (\max - \min)$	Scales data to a range between 0 and 1
Z-Score Standardization	$x' = (x - \mu) / \sigma$	Creates standardized features with mean 0 and standard deviation 1
Discretization (Binning)	pd.cut() or pd.qcut()	Converts continuous variables into categories
Label Encoding	LabelEncoder().fit_transform()	Converts categorical values into numeric labels

–

Data Preprocessing Procedure

Step 1: Load and Explore the Dataset

1. Import the required libraries:
   - Pandas
   - matplotlib
   - scikit-learn
2. Load the loan prediction dataset into a DataFrame.
3. Examine:
   - Dataset dimensions
   - Data types
   - Summary statistics
4. Identify numerical and categorical attributes.

Expected Outcome

The dataset is successfully loaded and ready for preprocessing.

Step 2: Apply Min-Max Normalization

1. Import MinMaxScaler from sklearn.preprocessing.
2. Select numerical features such as:
   - Income
   - Loan Amount
   - Credit Score
3. Apply Min-Max Scaling to transform values into the range [0,1].
4. Compare feature values before and after normalization.

Expected Outcome

All selected numerical attributes are scaled within a common range.

Step 3: Perform Z-Score Standardization

1. Import StandardScaler from sklearn.preprocessing.
2. Apply standardization to numerical attributes.
3. Calculate transformed values with: ○ Mean ≈ 0 ○ Standard Deviation ≈ 1
4. Observe the effect of standardization on feature distributions.

–

Expected Outcome

The transformed features have consistent statistical properties suitable for machine learning algorithms.

Step 4: Discretize Continuous Variables

1. Select continuous attributes such as Income or Age.
2. Use:
   - pd.cut() for equal-width intervals
   - pd.qcut() for equal-frequency intervals
3. Create meaningful categories such as: ○ Low ○ Medium ○ High
4. Analyze the resulting category distribution.

Expected Outcome

Continuous variables are successfully transformed into categorical groups.

Step 5: Encode Categorical Features

1. Import LabelEncoder from sklearn.preprocessing.
2. Apply Label Encoding to attributes such as:
3. Convert textual categories into numerical representations.
4. Verify successful encoding.

Expected Outcome

All categorical features are represented numerically and are suitable for machine learning models.

Step 6: Compare Data Distributions

1. Generate histograms before preprocessing.
2. Generate histograms after normalization and standardization.
3. Compare:
   - Feature ranges
   - Distribution shapes
   - Variance levels
4. Evaluate preprocessing effectiveness.

Expected Outcome

Improved feature consistency and more suitable distributions for predictive modeling.

–

Performance Evaluation Metrics

KPI	Purpose
Variance Reduction	Measures the impact of scaling on feature spread
Distribution Improvement	Evaluates how preprocessing improves feature consistency and model readiness

Expected Results

• Numerical features are successfully normalized and standardized.

• Continuous variables are converted into meaningful categories.

• Categorical variables are transformed into numerical form.

• Feature distributions become more consistent and comparable.

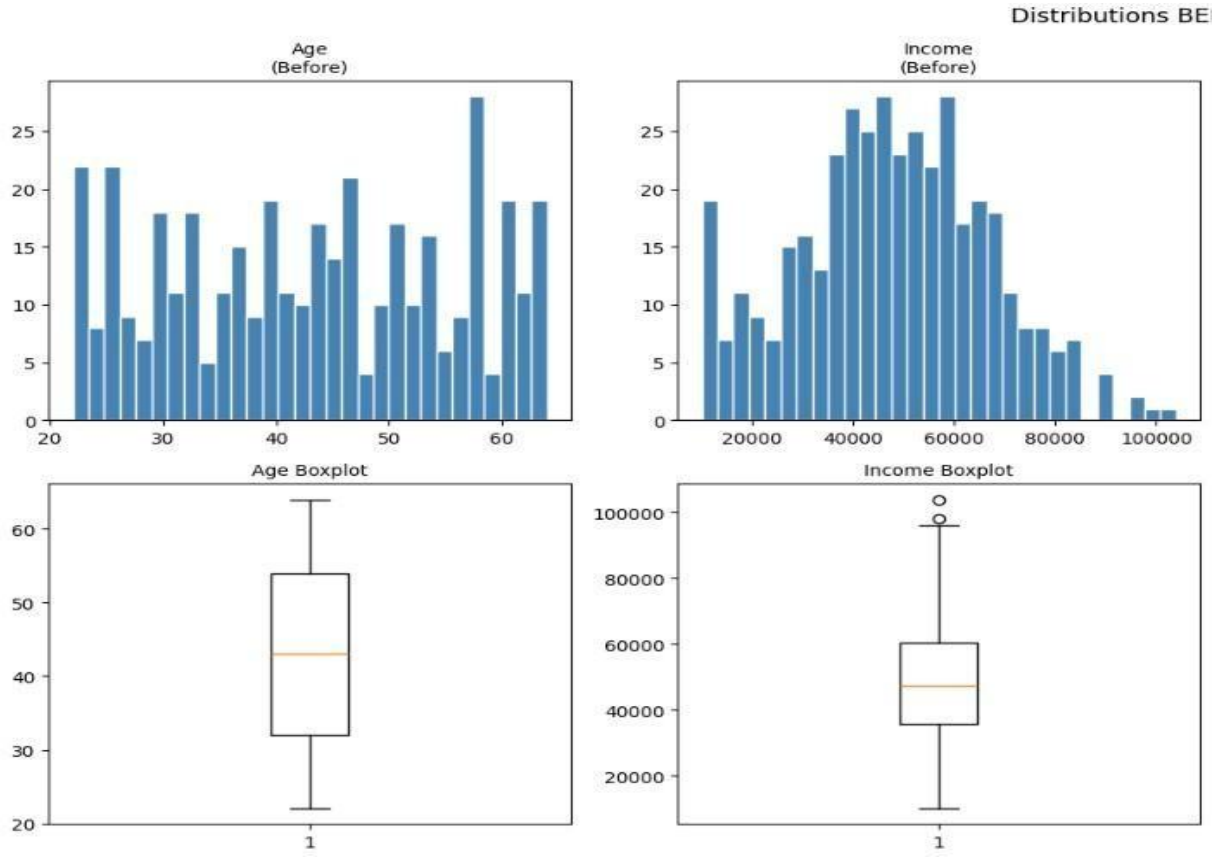
• The dataset becomes suitable for machine learning algorithms.

```
np.random.seed(0)
n4 = 400

df4 = pd.DataFrame({
    'ApplicantID':  range(1, n4+1),
    'Age':          np.random.randint(22, 65, n4),
    'Income':       np.random.normal(50000, 20000, n4).clip(10000),
    'LoanAmount':   np.random.normal(150000, 60000, n4).clip(10000),
    'CreditScore':  np.random.randint(300, 850, n4),
    'LoanTerm':     np.random.choice([12, 24, 36, 60, 120], n4),
    'Employment':   np.random.choice(['Salaried','Self-Employed','Business'], n4),
    'LoanStatus':   np.random.choice(['Approved','Rejected'], n4, p=[0.65,0.35])
})

num_cols4 = ['Age', 'Income', 'LoanAmount', 'CreditScore']
print('Shape:', df4.shape)
df4.head()
```

```
Shape: (400, 8)
```

```
print('=== Summary Statistics ===')
display(df4.describe().round(2))

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for i, col in enumerate(num_cols4):
    axes[0, i].hist(df4[col], bins=30, color='steelblue', edgecolor='white')
    axes[0, i].set_title(f'{col}\n(Before)', fontsize=10)
    axes[1, i].boxplot(df4[col])
    axes[1, i].set_title(f'{col} Boxplot', fontsize=10)
plt.suptitle('Distributions BEFORE Preprocessing', fontsize=13)
plt.tight_layout()
plt.show()
```

std	115.61	12.61	19156.66	57314.40	162.72	37.05
min	1.00	22.00	10000.00	10000.00	300.00	12.00
25%	100.75	32.00	35577.95	108935.72	427.75	24.00
50%	200.50	43.00	47517.29	148161.38	567.00	36.00
75%	300.25	54.00	60493.39	182671.62	706.25	60.00
max	400.00	64.00	103924.48	315561.31	849.00	120.00



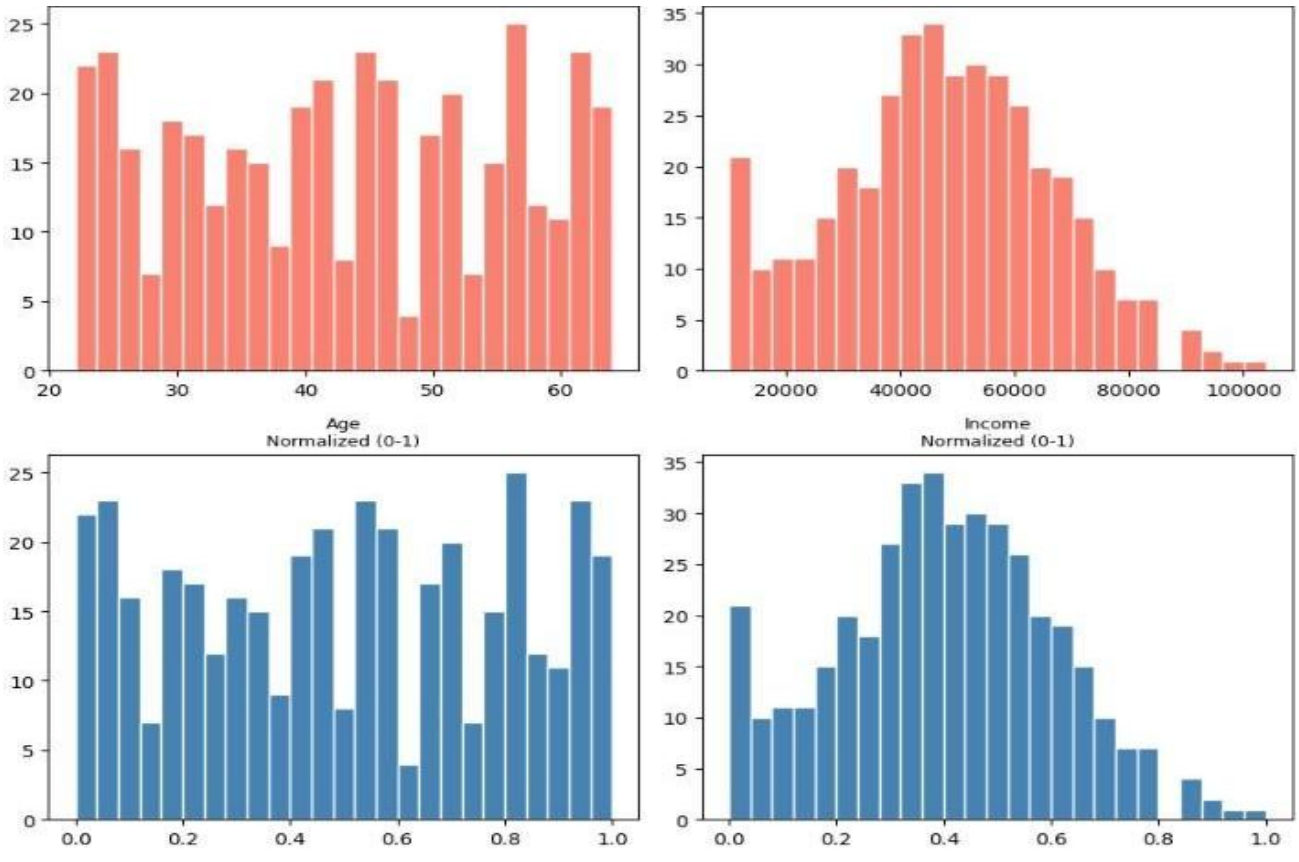
```
scaler_minmax = MinMaxScaler()
df_normalized = df4.copy()
df_normalized[num_cols4] = scaler_minmax.fit_transform(df4[num_cols4])

print('=== After Min-Max Normalization ===')
display(df_normalized[num_cols4].describe().round(4))

var_before = df4[num_cols4].var()
var_after  = df_normalized[num_cols4].var()
var_reduction = ((var_before - var_after) / var_before * 100)
print('\nVariance Reduction After Normalization:')
for col in num_cols4:
    print(f'  {col:<14}: {var_reduction[col]:.1f}% reduction')
```

```
=== After Min-Max Normalization ===
```

	Age	Income	LoanAmount	CreditScore
count	400.0000	400.0000	400.0000	400.0000
mean	0.4995	0.4001	0.4566	0.4846
std	0.3003	0.2040	0.1876	0.2964
min	0.0000	0.0000	0.0000	0.0000
25%	0.2381	0.2723	0.3238	0.2327
50%	0.5000	0.3994	0.4522	0.4863
75%	0.7619	0.5376	0.5651	0.7400
max	1.0000	1.0000	1.0000	1.0000

```
Variance Reduction After Normalization:
  Age           : 99.9% reduction
  Income        : 100.0% reduction
  LoanAmount    : 100.0% reduction
  CreditScore   : 100.0% reduction
```

```
scaler_std = StandardScaler()
df_standard = df4.copy()
df_standard[num_cols4] = scaler_std.fit_transform(df4[num_cols4])

print('=== After Standardization (Z-Score) ===')
display(df_standard[num_cols4].describe().round(4))

# Comparison plots
fig, axes = plt.subplots(3, 4, figsize=(18, 12))
for i, col in enumerate(num_cols4):
    axes[0, i].hist(df4[col],           bins=25, color='salmon',    edgecolor='white')
    axes[0, i].set_title(f'{col}\nOriginal', fontsize=9)
    axes[1, i].hist(df_normalized[col], bins=25, color='steelblue', edgecolor='white')
    axes[1, i].set_title(f'{col}\nNormalized (0-1)', fontsize=9)
    axes[2, i].hist(df_standard[col],   bins=25, color='seagreen',  edgecolor='white')
    axes[2, i].set_title(f'{col}\nStandardized (Z)', fontsize=9)
plt.suptitle('Distribution Comparison: Original vs Normalized vs Standardized', fontsize=13)
plt.tight_layout()
plt.show()
```



```
df_disc = df4.copy()
df_disc['AgeGroup']    = pd.cut(df4['Age'],         bins=[0,30,40,55,100],          labels=['Young','Mid','Senior','Elder'])
df_disc['IncomeLevel'] = pd.cut(df4['Income'],      bins=[0,30000,60000,100000,np.inf], labels=['Low','Medium','High','Very High'])
df_disc['CreditBand']  = pd.cut(df4['CreditScore'], bins=[0,579,669,739,799,850],    labels=['Poor','Fair','Good','Very Good','Exceptional'])

display(df_disc[['Age','AgeGroup','Income','IncomeLevel','CreditScore','CreditBand']].head(10))

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
df_disc['AgeGroup'].value_counts().sort_index().plot(kind='bar',   ax=axes[0], color='steelblue', edgecolor='black')
df_disc['IncomeLevel'].value_counts().sort_index().plot(kind='bar', ax=axes[1], color='salmon',    edgecolor='black')
df_disc['CreditBand'].value_counts().sort_index().plot(kind='bar',  ax=axes[2], color='seagreen',  edgecolor='black')
axes[0].set_title('Age Groups'); axes[1].set_title('Income Levels'); axes[2].set_title('Credit Score Bands')
plt.suptitle('Discretized Category Distributions', fontsize=13)
plt.tight_layout()
plt.show()
```

	Age	AgeGroup	Income	IncomeLevel	CreditScore	CreditBand
0	22	Young	40728.080507	Medium	783	Very Good
1	25	Young	59629.629475	Medium	747	Very Good
2	25	Young	19184.059711	Low	673	Good
3	61	Elder	51265.239884	Medium	343	Poor
4	31	Mid	53130.130759	Medium	455	Poor
5	41	Senior	54643.620724	Medium	658	Fair
6	43	Senior	38053.678621	Medium	835	Exceptional
7	58	Elder	45241.565405	Medium	590	Fair
8	45	Senior	21518.781820	Low	588	Fair
9	28	Young	40133.602333	Medium	578	Poor

```
df_encoded = df4.copy()
le = LabelEncoder()
for col in ['Employment', 'LoanStatus']:
    df_encoded[col + '_Code'] = le.fit_transform(df4[col])
    mapping = dict(zip(le.classes_, le.transform(le.classes_)))
    print(f'{col} encoding: {mapping}')

# Correlation Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(df_standard[num_cols4].corr(), annot=True, fmt='.2f', cmap='coolwarm', square=True, linewidths=0.5)
plt.title('Correlation Heatmap (Standardized Data)', fontsize=13)
plt.tight_layout()
plt.show()

print('\n=== LAB 4 - PREPROCESSING KPI REPORT ===')
print(f'{'Column':<16} {'Var Before':>14} {'Var Normalized':>16} {'Var Standardized':>18}')
print('-' * 66)
for col in num_cols4:
    vb = df4[col].var()
    vn = df_normalized[col].var()
    vs = df_standard[col].var()
    print(f'{col:<16} {vb:>14.2f} {vn:>16.6f} {vs:>18.4f}')

df_standard.to_csv('loan_preprocessed.csv', index=False)
print('\nSaved: loan_preprocessed.csv')
```

–

Lab No. 05: Market Basket Analysis

Topic

Association Rule Mining Using Manual Calculations

Business Scenario

A supermarket aims to improve its cross-selling strategy by identifying products that are frequently purchased together. Understanding customer buying patterns can help optimize product placement, promotions, and recommendation systems.

Dataset

Grocery Transactions Dataset (20 Transactions, 6 Items)

Software Used

Python (Manual Calculations and mlxtend Library)

Key Performance Indicators (KPIs)

• Frequent Itemsets

• High-Lift Association Rules

• Cross-Selling Opportunities

Objectives

• Understand the fundamentals of Market Basket Analysis.

• Calculate Support, Confidence, and Lift manually.

• Identify frequent itemsets from transaction data.

• Discover meaningful association rules among products.

• Validate manually calculated results using the mlxtend library.

Introduction

Market Basket Analysis is a data mining technique used to uncover relationships among items purchased together in customer transactions. Retailers use these insights to improve product recommendations, promotional campaigns, inventory planning, and store layouts.

–

Association Rule Mining identifies patterns such as customers who purchase one product are also likely to purchase another. These relationships are measured using metrics such as Support, Confidence, and Lift.

In this lab, students will manually calculate association measures and compare their results with outputs generated using Python libraries.

Dataset Description

The dataset consists of grocery shopping transactions containing combinations of six commonly purchased items.

Sample Products

• Bread

• Milk

• Butter

• Eggs

• Cheese • Juice

Each transaction represents the products purchased by a customer during a single shopping visit.

Fundamental Measures in Association Rule Mining

Measure	Formula	Purpose
Support	Count(A ∩ B) / Total Transactions	Indicates how frequently an itemset appears in the dataset
Confidence	Support(A ∩ B) / Support(A)	Measures the likelihood of purchasing B when A is purchased
Lift	Confidence(A → B) / Support(B)	Evaluates the strength of an association beyond random chance

```
Libraries loaded!
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

–

```python
transactions5 = [
    ["Milk","Bread","Butter"], ["Bread","Eggs"], ["Milk","Bread","Eggs","Butter"],
    ["Milk","Eggs"], ["Bread","Butter"], ["Milk","Bread"], ["Eggs","Butter"],
    ["Milk","Bread","Eggs"], ["Bread","Butter","Cheese"], ["Milk","Cheese"],
    ["Bread","Eggs","Cheese"], ["Milk","Butter","Cheese"], ["Milk","Bread","Butter","Cheese"],
    ["Eggs","Cheese"], ["Milk","Eggs","Cheese"], ["Bread","Milk"],
    ["Butter","Eggs"], ["Bread","Cheese"], ["Milk","Eggs"], ["Bread","Butter","Eggs"],
]
N5 = len(transactions5)
print(f"Total Transactions: {N5}")
print('Sample:', transactions5[:3])
```

```
Total Transactions: 20
Sample: [['Milk', 'Bread', 'Butter'], ['Bread', 'Eggs'], ['Milk', 'Bread', 'Eggs', 'Butter']]
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

–

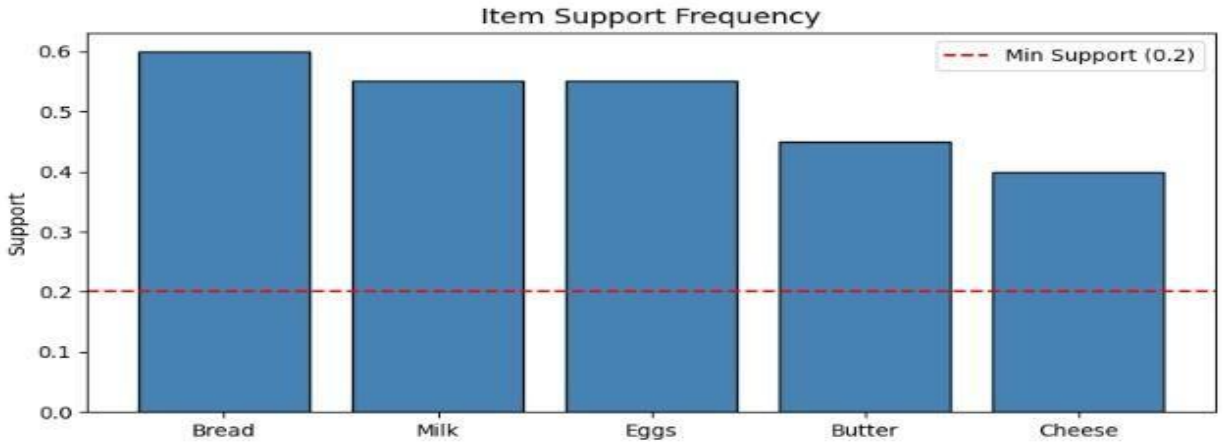
```
item_counts5 = defaultdict(int)
for t in transactions5:
    for item in t:
        item_counts5[item] += 1

support_1 = {item: count/N5 for item, count in item_counts5.items()}

print('=== Single Item Support ===')
df_sup = pd.DataFrame({'Item': list(support_1.keys()), 'Count': list(item_counts5.values()),
                       'Support': [round(v,3) for v in support_1.values()]
                      }).sort_values('Count', ascending=False)
print(df_sup.to_string(index=False))
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
=== Single Item Support ===
  Item  Count  Support
 Bread     12     0.60
  Milk     11     0.55
  Eggs     11     0.55
Butter      9     0.45
Cheese      8     0.40
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.utcnow().replace(tzinfo=utc)
```

```
def sup(items, txns):  return sum(1 for t in txns if set(items).issubset(set(t))) / len(txns)
def conf(A, B, txns):  sa = sup(A, txns); return sup(list(A)+list(B), txns)/sa if sa>0 else 0
def lift(A, B, txns):  sb = sup(B, txns); return conf(A,B,txns)/sb if sb>0 else 0

MIN_SUPPORT5 = 0.20
pair_counts5 = defaultdict(int)
for t in transactions5:
    for pair in combinations(sorted(set(t)), 2):
        pair_counts5[pair] += 1
freq_pairs5 = {p: c/N5 for p, c in pair_counts5.items() if c/N5 >= MIN_SUPPORT5}

rules5 = []
for (i1, i2) in freq_pairs5:
    for A, B in [([i1],[i2]),([i2],[i1])]:
        rules5.append({'Rule': f'{A[0]} → {B[0]}', 'Support': round(sup(A+B,transactions5),3),
                       'Confidence': round(conf(A,B,transactions5),3), 'Lift': round(lift(A,B,transactions5),3)})

df_rules5 = pd.DataFrame(rules5).sort_values('Lift', ascending=False)
print('=== Manual Association Rules ===')
print(df_rules5.to_string(index=False))
```

```
=== Manual Association Rules ===
          Rule  Support  Confidence  Lift
Bread → Butter     0.30       0.500 1.111
Butter → Bread     0.30       0.667 1.111
  Bread → Milk     0.30       0.500 0.909
  Milk → Bread     0.30       0.545 0.909
 Milk → Cheese     0.20       0.364 0.909
 Cheese → Milk     0.20       0.500 0.909
Cheese → Bread     0.20       0.500 0.833
Bread → Cheese     0.20       0.333 0.833
   Eggs → Milk     0.25       0.455 0.826
   Milk → Eggs     0.25       0.455 0.826
 Milk → Butter     0.20       0.364 0.808
 Butter → Milk     0.20       0.444 0.808
 Butter → Eggs     0.20       0.444 0.808
 Eggs → Butter     0.20       0.364 0.808
  Eggs → Bread     0.25       0.455 0.758
  Bread → Eggs     0.25       0.417 0.758
```

```
items_s = sorted(support_1.items(), key=lambda x: x[1], reverse=True)
plt.figure(figsize=(8, 4))
plt.bar([x[0] for x in items_s], [x[1] for x in items_s], color='steelblue', edgecolor='black')
plt.axhline(y=MIN_SUPPORT5, color='red', linestyle='--', label=f'Min Support ({MIN_SUPPORT5})')
plt.ylabel('Support'); plt.title('Item Support Frequency', fontsize=13); plt.legend()
plt.tight_layout(); plt.show()

plt.figure(figsize=(9, 6))
sc = plt.scatter(rules5_lib['support'], rules5_lib['confidence'], c=rules5_lib['lift'], cmap='RdYlGn', s=100, edgecolors='grey')
plt.colorbar(sc, label='Lift'); plt.xlabel('Support'); plt.ylabel('Confidence')
plt.title('Association Rules: Support vs Confidence (Color = Lift)', fontsize=13)
plt.tight_layout(); plt.show()

high_lift5 = rules5_lib[rules5_lib['lift'] > 1.2]
print('=' * 45)
print('  LAB 5 – MARKET BASKET KPI REPORT')
print('=' * 45)
print(f'Total Transactions         : {N5}')
print(f'Frequent Itemsets Found    : {len(freq5)}')
print(f'Total Rules                : {len(rules5_lib)}')
print(f'High Lift Rules (> 1.2)    : {len(high_lift5)}')
print(f'Cross-Sell Pairs (conf≥0.6): {(rules5_lib["confidence"]>=0.6).sum()}')
print('=' * 45)
```



–

Lab No. 06: Apriori Algorithm Implementation

Topic

Apriori Algorithm – Manual and Automated Implementation

Business Scenario

A supermarket wants to analyze customer purchasing behavior by identifying groups of products that are frequently purchased together. The findings can be used to improve product placement, promotions, and recommendation strategies.

Dataset

Transaction Dataset (20 Transactions, 8 Items)

Software Used

Python (Custom Apriori Implementation and mlxtend Library)

Key Performance Indicators (KPIs)

• Number of Generated Association Rules

• Impact of Minimum Support Threshold

Objectives

• Understand the working principles of the Apriori Algorithm.

• Generate frequent itemsets using manual implementation techniques.

• Apply the Apriori Algorithm using Python libraries.

• Analyze the influence of support thresholds on discovered patterns.

• Generate and evaluate association rules from frequent itemsets.

Introduction

The Apriori Algorithm is one of the most widely used techniques for association rule mining and frequent pattern discovery. It identifies combinations of items that occur together frequently within transactional datasets.

–

The algorithm relies on the Apriori Principle, which states that if an itemset is frequent, then all of its subsets must also be frequent. This property significantly reduces the search space by eliminating candidate itemsets that cannot satisfy the minimum support requirement.

Apriori is commonly used in retail analytics, recommendation systems, market basket analysis, and customer behavior studies.

Dataset Description

The dataset contains 20 shopping transactions involving 8 different products. Each transaction records the items purchased by a customer during a single visit.

Sample Product Categories

• Bread

• Milk

• Eggs

• Butter

• Cheese

• Juice

• Coffee

• Snacks

The objective is to discover frequently occurring product combinations and derive meaningful association rules.

Apriori Algorithm Overview

The Apriori Algorithm follows an iterative process to generate frequent itemsets of increasing size.

Working Principle

The algorithm begins by identifying frequent single-item sets. It then combines these frequent itemsets to generate larger candidate itemsets while removing combinations that fail to meet the minimum support requirement.

This process continues until no additional frequent itemsets can be found.

–

Apriori Workflow

Stage	Description
Candidate Generation (C1)	Generate all possible single-item candidates
Frequent Itemsets (L1)	Retain only items meeting minimum support
Candidate Generation (C2)	Combine frequent 1-itemsets to form 2-item candidates
Frequent Itemsets (L2)	Remove candidates below support threshold
Candidate Expansion	Continue generating larger itemsets iteratively
Rule Generation	Produce association rules using confidence criteria

Step 1 – Dataset

```
transactions6 = [
    ['Bread','Milk','Butter'], ['Bread','Diaper','Beer','Egg'], ['Milk','Diaper','Beer','Cola'],
    ['Bread','Milk','Diaper','Beer'], ['Bread','Milk','Diaper','Cola'], ['Bread','Milk'],
    ['Milk','Diaper'], ['Beer','Cola'], ['Bread','Butter','Egg'], ['Milk','Butter','Egg'],
    ['Bread','Diaper'], ['Diaper','Beer','Cola'], ['Bread','Milk','Egg'], ['Milk','Cola'],
    ['Bread','Diaper','Beer'], ['Bread','Milk','Butter','Egg'], ['Diaper','Cola'],
    ['Beer','Egg'], ['Milk','Bread','Diaper'], ['Butter','Egg','Cola'],
]

all_items6 = sorted(set(item for t in transactions6 for item in t))
print(f'Transactions: {len(transactions6)} | Unique Items: {all_items6}')
```

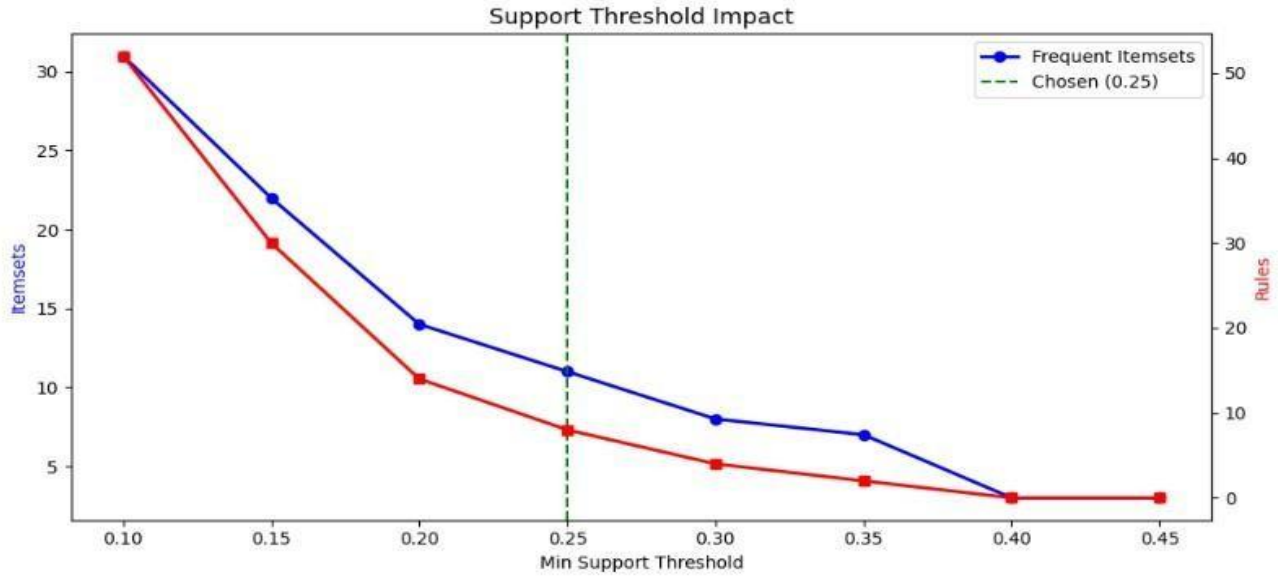
```
Transactions: 20 | Unique Items: ['Beer', 'Bread', 'Butter', 'Cola', 'Diaper', 'Egg', 'Milk']
```

```
class AprioriManual:
    def __init__(self, transactions, min_support):
        self.transactions = [set(t) for t in transactions]
        self.N = len(transactions)
        self.min_support = min_support
        self.freq_itemsets = {}

    def _get_support(self, itemset):
        fs = frozenset(itemset)
        return sum(1 for t in self.transactions if fs.issubset(t)) / self.N

    def _generate_candidates(self, prev_frequent, k):
        candidates = set()
        items = list(prev_frequent)
        for i in range(len(items)):
            for j in range(i+1, len(items)):
                union = items[i] | items[j]
                if len(union) == k:
                    candidates.add(union)
        return candidates

    def fit(self):
        all_items = set(item for t in self.transactions for item in t)
        C1 = [frozenset([item]) for item in all_items]
        L1 = {c: self._get_support(c) for c in C1 if self._get_support(c) >= self.min_support}
        self.freq_itemsets.update(L1)
        current_L = set(L1.keys()); k = 2
        while current_L:
            candidates = self._generate_candidates(current_L, k)
            Lk = {c: self._get_support(c) for c in candidates if self._get_support(c) >= self.min_support}
            if not Lk: break
            self.freq_itemsets.update(Lk)
            current_L = set(Lk.keys()); k += 1
        return self

    def summary(self):
        return pd.DataFrame([{'Itemset': str(set(is_)), 'Size': len(is_), 'Support': round(s,4)}
                             for is_, s in sorted(self.freq_itemsets.items(), key=lambda x:-x[1])])

MIN_SUPPORT6 = 0.25
apr6 = AprioriManual(transactions6, MIN_SUPPORT6).fit()
df_freq6 = apr6.summary()
print(f'Frequent Itemsets (min_support={MIN_SUPPORT6}): {len(df_freq6)}')
print(df_freq6.to_string(index=False))
```

```
MIN_CONF6 = 0.5
rules6 = []
for itemset, s in apr6.freq_itemsets.items():
    if len(itemset) < 2: continue
    for i in range(1, len(itemset)):
        for ant in combinations(itemset, i):
            ant = frozenset(ant); con = itemset - ant
            s_ant = apr6.freq_itemsets.get(ant, apr6._get_support(ant))
            s_con = apr6.freq_itemsets.get(con, apr6._get_support(con))
            c = s/s_ant if s_ant>0 else 0
            l = c/s_con if s_con>0 else 0
            if c >= MIN_CONF6:
                rules6.append({'Antecedent':str(set(ant)),'Consequent':str(set(con)),
                               'Support':round(s,4),'Confidence':round(c,4),'Lift':round(l,4)})

df_rules6 = pd.DataFrame(rules6).sort_values('Lift', ascending=False).reset_index(drop=True)
print(f'Association Rules (min_conf={MIN_CONF6}): {len(df_rules6)}')
display(df_rules6)

# mlxtend validation
te6 = TransactionEncoder()
df_enc6 = pd.DataFrame(te6.fit_transform(transactions6), columns=te6.columns_)
freq6_lib = mlx_apriori(df_enc6, min_support=MIN_SUPPORT6, use_colnames=True)
rules6_lib = mlx_rules(freq6_lib, metric='confidence', min_threshold=MIN_CONF6)
print(f'mlxtend: {len(freq6_lib)} itemsets, {len(rules6_lib)} rules')
```

```
...  Association Rules (min_conf=0.5): 7
```

```
thresholds6 = [0.10,0.15,0.20,0.25,0.30,0.35,0.40,0.45]
impact6 = []
for t in thresholds6:
    fi = mlx_apriori(df_enc6, min_support=t, use_colnames=True)
    nr = len(mlx_rules(fi, metric='confidence', min_threshold=0.3)) if len(fi) > 0 else 0
    impact6.append({'min_support': t, 'freq_itemsets': len(fi), 'rules': nr})

df_impact6 = pd.DataFrame(impact6)
fig, ax1 = plt.subplots(figsize=(10, 5))
ax1.plot(df_impact6['min_support'], df_impact6['freq_itemsets'], 'b-o', linewidth=2, label='Frequent Itemsets')
ax1.set_xlabel('Min Support Threshold'); ax1.set_ylabel('Itemsets', color='blue')
ax2 = ax1.twinx()
ax2.plot(df_impact6['min_support'], df_impact6['rules'], 'r-s', linewidth=2, label='Rules')
ax2.set_ylabel('Rules', color='red')
ax1.axvline(x=MIN_SUPPORT6, color='green', linestyle='--', label=f'Chosen ({MIN_SUPPORT6})')
ax1.legend(loc='upper right'); plt.title('Support Threshold Impact', fontsize=13)
plt.tight_layout(); plt.show()

print('=' * 45)
print('   LAB 6 - APRIORI KPI REPORT')
print('=' * 45)
print(f'Frequent Itemsets     : {len(df_freq6)}')
print(f'Association Rules     : {len(df_rules6)}')
print(f'High Lift (> 1.5)     : {(df_rules6["Lift"] > 1.5).sum()}')
print(f'Max Lift              : {df_rules6["Lift"].max():.3f}')
print('=' * 45)
```



–

LAB 07: FP-GROWTH ALGORITHM AND PERFORMANCE COMPARISON

Topic

FP-Growth vs Apriori – Efficiency Analysis in Frequent Pattern Mining

Business Scenario

A large retail organization aims to discover frequent purchasing patterns from a high-volume transaction dataset. The objective is to evaluate efficient mining techniques for association rule mining and compare their computational performance.

Dataset

Synthetic Large Retail Dataset (1,000 Transactions, 20 Items)

Software Used

Python (mlxtend – FP-Growth, Matplotlib)

Key Performance Indicators (KPIs)

- Execution Time
- Number of Generated Rules
- Speedup Factor

Objectives

- Understand the working mechanism of the FP-Growth algorithm
- Compare FP-Growth with Apriori in terms of efficiency and scalability
- Extract frequent itemsets from large transactional datasets
- Measure and analyze execution time differences
- Evaluate algorithm performance using speedup factor

Introduction

FP-Growth (Frequent Pattern Growth) is an advanced frequent itemset mining algorithm designed to overcome the limitations of Apriori. Instead of generating candidate itemsets repeatedly, FP-Growth compresses the dataset into a compact structure called an FP-Tree and extracts frequent patterns directly.
This approach significantly reduces computational cost and improves performance, especially for large datasets with high dimensionality.

–

In this lab, FP-Growth is implemented and compared with Apriori to evaluate efficiency and scalability.

FP-Growth Algorithm Overview

FP-Growth is a pattern mining technique that avoids candidate generation by using a tree-based representation of transactions.

Key Features
• Eliminates candidate generation
• Requires only two database scans
• Uses FP-Tree for compact data representation
• Highly efficient for large datasets

Comparison: Apriori vs FP-Growth

Aspect	Apriori	FP-Growth
Approach	Candidate generation and pruning	FP-Tree-based pattern extraction
Database Scans	Multiple scans (per iteration)	Only two scans
Memory Usage	High due to candidate storage	Low due to compressed tree structure
Execution Speed	Slower	Faster
Scalability	Poor for large datasets	Highly scalable
Implementation	Simple	More complex

Procedure

Step 1: Load and Prepare Dataset

• Import required Python libraries
• Load the transaction dataset
• Perform one-hot encoding on transactional data
• Verify dataset structure

Expected Outcome
Data is successfully transformed into a format suitable for FP-Growth analysis

Step 2: Apply FP-Growth Algorithm

• Import FP-Growth from mlxtend
• Set minimum support threshold
• Generate frequent itemsets
• Extract patterns from transactions

–

Expected Outcome
Frequent itemsets are generated efficiently without candidate generation

Step 3: Generate Association Rules

• Apply association_rules() function
• Compute support, confidence, and lift
• Filter rules using thresholds

Expected Outcome
Strong association rules are successfully extracted

Step 4: Performance Evaluation

• Measure execution time of FP-Growth
• Compare with Apriori execution time
• Compute speedup factor
• Visualize results using Matplotlib

Speedup Formula
Speedup = Apriori Time / FP-Growth Time

Expected Outcome
FP-Growth demonstrates superior performance in terms of speed and efficiency

Performance Metrics

KPI	Description
Execution Time	Time required to generate frequent itemsets and rules
Number of Rules	Total association rules generated
Speedup Factor	Relative performance improvement over Apriori

Expected Results

• FP-Growth executes faster than Apriori on large datasets
• Memory usage is significantly reduced due to FP-Tree structure
• Both algorithms produce similar association rules
• FP-Growth scales better with increasing dataset size

```
import time
from mlxtend.frequent_patterns import fpgrowth

np.random.seed(42)
ITEMS7 = ['Bread','Milk','Butter','Eggs','Cheese','Beer','Cola','Juice','Tea','Coffee',
          'Chips','Cookies','Cereal','Rice','Pasta','Chicken','Fish','Beef','Yogurt','Apple']
probs7 = [0.6,0.6,0.35,0.45,0.3,0.35,0.3,0.25,0.3,0.35,0.2,0.25,0.3,0.4,0.3,0.4,0.3,0.3,0.25,0.35]
N7 = 1000
transactions7 = [[item for item,p in zip(ITEMS7,probs7) if np.random.random()<p] for _ in range(N7)]
transactions7 = [t for t in transactions7 if t]

te7 = TransactionEncoder()
df_enc7 = pd.DataFrame(te7.fit_transform(transactions7), columns=te7.columns_)
print(f'Generated {len(transactions7)} transactions | Avg basket: {np.mean([len(t) for t in transactions7]):.2f}')
```

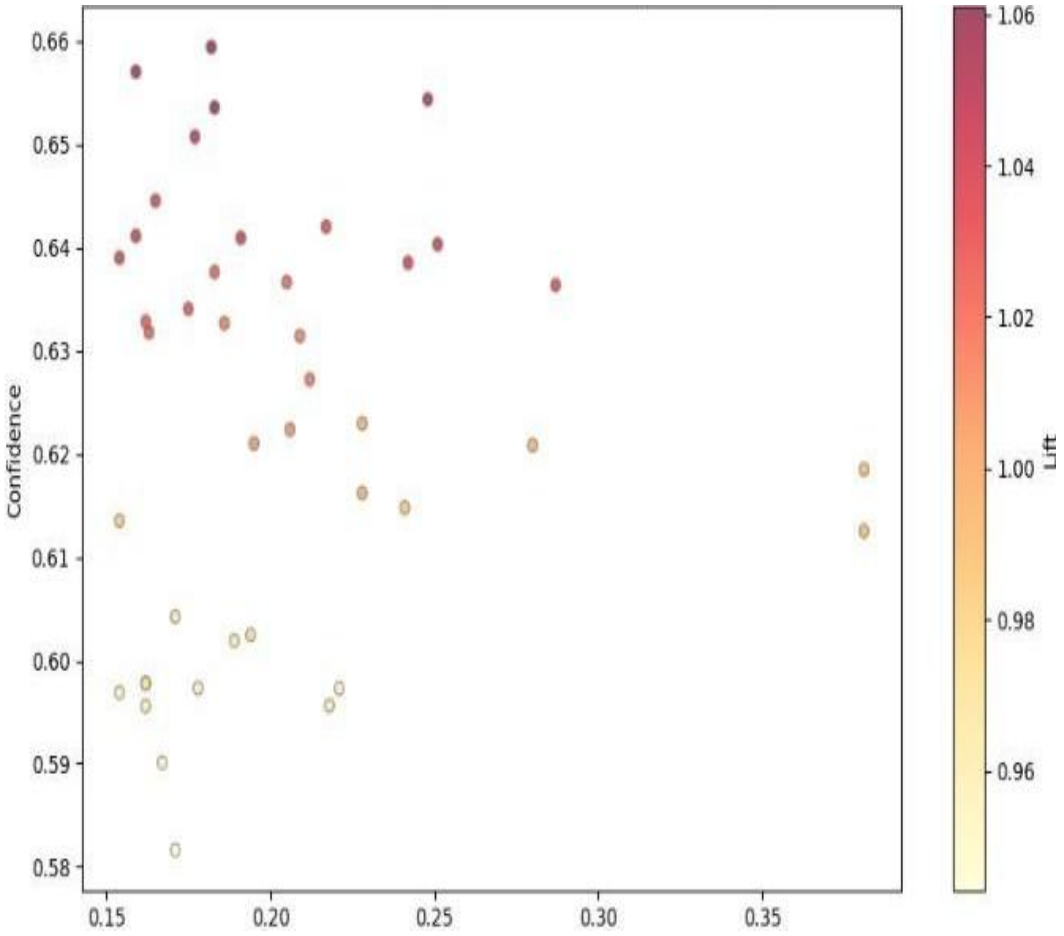
```
Generated 1000 transactions | Avg basket: 6.93
```

```
fig, ax = plt.subplots(figsize=(10, 7))
ax.set_xlim(0,10); ax.set_ylim(0,8); ax.axis('off')

nodes7 = {
    'root':      (5.0,7.0,'Root\n{}'),
    'bread':     (3.5,5.5,'Bread\n(4)'),
    'milk_b':    (2.5,4.0,'Milk\n(3)'),
    'milk_a':    (5.5,5.5,'Milk\n(2)'),
    'butter_bm': (1.5,2.5,'Butter\n(2)'),
    'eggs_bm':   (3.5,2.5,'Eggs\n(1)'),
    'butter_b':  (5.0,4.0,'Butter\n(1)'),
    'eggs_m':    (6.5,4.0,'Eggs\n(1)'),
}
edges7 = [('root','bread'),('root','milk_a'),('bread','milk_b'),('bread','butter_b'),
          ('milk_b','butter_bm'),('milk_b','eggs_bm'),('milk_a','eggs_m')]
clr7 = {'root':'#95a5a6','bread':'#3498db','milk_b':'#e74c3c','milk_a':'#e74c3c',
        'butter_bm':'#2ecc71','eggs_bm':'#f39c12','butter_b':'#2ecc71','eggs_m':'#f39c12'}

for src,dst in edges7:
    x1,y1,_ = nodes7[src]; x2,y2,_ = nodes7[dst]
    ax.annotate('', xy=(x2,y2+0.25), xytext=(x1,y1-0.25), arrowprops=dict(arrowstyle='->',color='gray',lw=1.5))
for k,(x,y,lbl) in nodes7.items():
    ax.text(x,y,lbl,ha='center',va='center',fontsize=10,fontweight='bold',
            bbox=dict(boxstyle='round,pad=0.4',facecolor=clr7.get(k,'lightblue'),edgecolor='black',alpha=0.9))

ax.set_title('FP-Tree Structure (support count in parentheses)', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()
```

```
from mlxtend.frequent_patterns import apriori as mlx_apr2
MIN_SUPPORT7 = 0.15

t0 = time.time(); freq_apr7 = mlx_apr2(df_enc7, min_support=MIN_SUPPORT7, use_colnames=True); time_apr7 = time.time()-t0
rules_apr7 = mlx_rules(freq_apr7, metric='confidence', min_threshold=0.5)

t0 = time.time(); freq_fpg7 = fpgrowth(df_enc7, min_support=MIN_SUPPORT7, use_colnames=True); time_fpg7 = time.time()-t0
rules_fpg7 = mlx_rules(freq_fpg7, metric='confidence', min_threshold=0.5)

print(f'{'Algorithm':<12} {'Time(s)':>10} {'Itemsets':>10} {'Rules':>8}')
print('-'*42)
print(f'{'Apriori':<12} {time_apr7:>10.4f} {len(freq_apr7):>10} {len(rules_apr7):>8}')
print(f'{'FP-Growth':<12} {time_fpg7:>10.4f} {len(freq_fpg7):>10} {len(rules_fpg7):>8}')
if time_fpg7>0: print(f'\nSpeedup: {time_apr7/time_fpg7:.2f}x')
```

```
Algorithm       Time(s)   Itemsets    Rules
------------------------------------------
Apriori          0.0086         65       42
FP-Growth        0.1752         65       42

Speedup: 0.05x
```

```
sizes7 = [100,200,400,600,800,1000]; t_apr7=[]; t_fpg7=[]
for sz in sizes7:
    te2=TransactionEncoder(); arr2=te2.fit_transform(transactions7[:sz]); df2=pd.DataFrame(arr2,columns=te2.columns_)
    t0=time.time(); mlx_apr2(df2,min_support=MIN_SUPPORT7,use_colnames=True); t_apr7.append(time.time()-t0)
    t0=time.time(); fpgrowth(df2,min_support=MIN_SUPPORT7,use_colnames=True); t_fpg7.append(time.time()-t0)

plt.figure(figsize=(9,5))
plt.plot(sizes7,t_apr7,'b-o',label='Apriori',linewidth=2)
plt.plot(sizes7,t_fpg7,'r-s',label='FP-Growth',linewidth=2)
plt.xlabel('Number of Transactions'); plt.ylabel('Execution Time (s)')
plt.title('Apriori vs FP-Growth: Execution Time Comparison',fontsize=13)
plt.legend(); plt.grid(True,alpha=0.3); plt.tight_layout(); plt.show()

# Rules scatter
plt.figure(figsize=(9,6))
sc=plt.scatter(rules_fpg7['support'],rules_fpg7['confidence'],c=rules_fpg7['lift'],
               cmap='YlOrRd',s=rules_fpg7['lift']*30,alpha=0.7,edgecolors='grey')
plt.colorbar(sc,label='Lift'); plt.xlabel('Support'); plt.ylabel('Confidence')
plt.title('FP-Growth Rules: Support vs Confidence (Bubble=Lift)',fontsize=12)
plt.tight_layout(); plt.show()

print('='*50)
print('  LAB 7 - FP-GROWTH vs APRIORI KPI REPORT')
print('='*50)
print(f'{'Metric':<25} {'Apriori':>10} {'FP-Growth':>12}')
print('-'*50)
print(f'{'Exec Time (s)':<25} {time_apr7:>10.4f} {time_fpg7:>12.4f}')
print(f'{'Frequent Itemsets':<25} {len(freq_apr7):>10} {len(freq_fpg7):>12}')
print(f'{'Association Rules':<25} {len(rules_apr7):>10} {len(rules_fpg7):>12}')
print(f'{'Max Lift':<25} {rules_apr7["lift"].max():>10.3f} {rules_fpg7["lift"].max():>12.3f}')
print('='*50)
```


0.66
0.65
0.64
0.63
0.62
0.61
0.60
0.59
0.58
Confidence
0.15
0.20
0.25
0.30
0.35
1.06
1.04
1.02
1.00
0.98
0.96
Lift

–

LAB 08: PYTHON FUNDAMENTALS AND REUSABLE DATA PROCESSING MODULE

Topic

Python Fundamentals and Object-Oriented Programming for Data Science

Business Scenario

A data analytics team needs a reusable preprocessing module to handle customer datasets efficiently. The goal is to design a structured Python pipeline that can be reused across multiple projects with minimal code changes.

Dataset

Synthetic Customer Dataset (200 Rows)

Software Used

Python (Built-in Libraries, pandas, scikit-learn)

Key Performance Indicators (KPIs)

- Code Efficiency
- Reusability of Components
- Number of Pipeline Steps
- Execution Time

Objectives

- Apply Python lists and list comprehensions for data manipulation tasks
- Use dictionaries for structured data storage and retrieval
- Develop reusable functions with proper documentation (docstrings)
- Design an object-oriented DataProcessor class with method chaining
- Execute a complete preprocessing pipeline and evaluate performance

Introduction

Python is widely used in data science due to its simplicity, flexibility, and extensive ecosystem of libraries. Efficient data processing often requires reusable and modular code structures.

–

Object-Oriented Programming (OOP) enables developers to organize code into classes and objects, improving scalability and maintainability. In this lab, a reusable data processing module is created using both functional programming and OOP principles.

The focus is on building a structured pipeline that can be applied to different datasets without rewriting core logic.

Dataset Description

The synthetic customer dataset contains structured information about customer behavior and attributes used for data analysis and preprocessing tasks.

Sample Attributes

- Customer ID – Unique identifier for each customer
- Age – Customer age
- Gender – Gender category
- Income – Annual income
- Spending Score – Customer spending behavior index
- Region – Geographic region
- Purchase Frequency – Number of purchases made

Python Concepts Used

Lists and List Comprehensions

Used for efficient iteration, filtering, and transformation of dataset values.

Dictionaries

Used for structured storage of key-value data representations.

Functions

Reusable functions are created with docstrings for clarity and maintainability.

Object-Oriented Programming

A custom class named DataProcessor is developed to encapsulate preprocessing operations.

Data Processing Pipeline

Step 1: Data Loading and Inspection

- Load dataset using pandas

–

• Display summary statistics
• Check for missing values and data types

Expected Outcome
Dataset is successfully loaded and validated

Step 2: Data Cleaning Functions

• Handle missing values using imputation
• Remove duplicate records
• Standardize column formats

Expected Outcome
Clean and consistent dataset is obtained

Step 3: Feature Engineering Using Python

• Apply list comprehensions for transformations
• Create derived features using dictionary mappings
• Encode categorical variables

Expected Outcome
Enhanced dataset with improved feature representation

Step 4: Object-Oriented Module Design

- Create DataProcessor class
- Implement methods for:

- load_data()
- clean_data()
- transform_data()
- export_data()

- Enable method chaining for streamlined execution

Expected Outcome
Reusable preprocessing class is successfully implemented

Step 5: Pipeline Execution

• Run full preprocessing pipeline
• Measure execution time
• Evaluate number of processing steps executed
Expected Outcome
End-to-end pipeline runs successfully with performance tracking

–

Performance Evaluation

KPI	Description
Code Efficiency	Measures simplicity and optimization of implementation
Reusability	Ability to reuse module across datasets
Pipeline Steps	Number of automated processing stages
Execution Time	Time taken to complete full preprocessing workflow

Expected Results

• Efficient Python-based preprocessing pipeline is developed
• Functions and class-based modules improve reusability
• Data processing steps are modular and well-structured
• Execution time is optimized compared to manual processing
• The DataProcessor class supports scalable data workflows

```
sales8 = [4500,7800,3200,9100,5600,8800,4100,6700]
months8 = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug']

print('Sales Data:', sales8)
print(f'Total: {sum(sales8):,}  |  Average: {sum(sales8)/len(sales8):,.0f}')
print(f'Max: {max(sales8):,}    |  Min: {min(sales8):,}')

above_avg8 = [s for s in sales8 if s > sum(sales8)/len(sales8)]
print('\nAbove Average:', above_avg8)
print('Top 3 Months:', sorted(zip(sales8,months8), reverse=True)[:3])

print('\nMonthly Sales:')
for i,(m,s) in enumerate(zip(months8,sales8),1):
    bar = '' * (s//500)
    print(f'  {i}. {m}: ${s:,} {bar}')
```

```
Sales Data: [4500, 7800, 3200, 9100, 5600, 8800, 4100, 6700]
Total: 49,800  |  Average: 6,225
Max: 9,100    |  Min: 3,200

Above Average: [7800, 9100, 8800, 6700]
Top 3 Months: [(9100, 'Apr'), (8800, 'Jun'), (7800, 'Feb')]
```

```
customer8 = {'id':'C001','name':'Alice Johnson','age':32,
             'email':'alice@example.com','purchases':[120,340,75,890,215]}

print('Customer Info:')
for k,v in customer8.items(): print(f'  {k:<12}: {v}')
print(f'\nTotal Spent: ${sum(customer8["purchases"]):,}')

products8 = {
    'P001':{'name':'Laptop',    'price':999, 'category':'Electronics','stock':50},
    'P002':{'name':'Headphones','price':149, 'category':'Electronics','stock':200},
    'P003':{'name':'Desk Chair','price':299, 'category':'Furniture',  'stock':80},
    'P004':{'name':'Notebook',  'price':9,   'category':'Stationery', 'stock':500},
    'P005':{'name':'Monitor',   'price':349, 'category':'Electronics','stock':120},
}

rev_cat = {}
for info in products8.values():
    rev_cat[info['category']] = rev_cat.get(info['category'],0) + info['price']*info['stock']
print('\nRevenue Potential by Category:')
for cat,rev in sorted(rev_cat.items(), key=lambda x:-x[1]):
    print(f'  {cat:<15}: ${rev:,}')
```

```
Customer Info:
  id           : C001
  name         : Alice Johnson
  age          : 32
  email        : alice@example.com
  purchases    : [120, 340, 75, 890, 215]

Total Spent: $1,640

Revenue Potential by Category:
  Electronics    : $121,630
  Furniture      : $23,920
  Stationery     : $4,500
```

```
def calculate_stats(data):
    """Return basic statistics for a numeric list."""
    if not data: return {}
    n = len(data); mean = sum(data)/n
    s = sorted(data)
    median = s[n//2] if n%2!=0 else (s[n//2-1]+s[n//2])/2
    std = (sum((x-mean)**2 for x in data)/n)**0.5
    return {'count':n,'mean':round(mean,2),'median':round(median,2),'std':round(std,2),'min':min(data),'max':max(data)}

def normalize(data):
    """Min-Max normalization → [0,1]"""
    mn,mx = min(data),max(data)
    return [round((x-mn)/(mx-mn),4) for x in data] if mx!=mn else [0.0]*len(data)

def standardize(data):
    """Z-score standardization"""
    st = calculate_stats(data); m,sd = st['mean'],st['std']
    return [round((x-m)/sd,4) for x in data] if sd!=0 else [0.0]*len(data)

def remove_outliers_iqr(data):
    """IQR outlier removal"""
    s=sorted(data); n=len(s); q1=s[n//4]; q3=s[3*n//4]; iqr=q3-q1
    return [x for x in data if (q1-1.5*iqr)<=x<=(q3+1.5*iqr)]

test_data8 = [23,45,12,67,34,89,11,54,200,41]
print('Original:', test_data8)
print('Stats:   ', calculate_stats(test_data8))
print('Normalized:', normalize(test_data8))
print('Standardized:', standardize(test_data8))
print('IQR Cleaned:', remove_outliers_iqr(test_data8))
```

```
Original: [23, 45, 12, 67, 34, 89, 11, 54, 200, 41]
Stats:    {'count': 10, 'mean': 57.6, 'median': 43.0, 'std': 52.73, 'min': 11, 'max': 200}
Normalized: [0.0635, 0.1799, 0.0053, 0.2963, 0.1217, 0.4127, 0.0, 0.2275, 1.0, 0.1587]
Standardized: [-0.6562, -0.239, -0.8648, 0.1783, -0.4476, 0.5955, -0.8837, -0.0683, 2.7005, -0.3148]
IQR Cleaned: [23, 45, 12, 67, 34, 89, 11, 54, 41]
```

```
class DataProcessor:
    """Reusable data preprocessing pipeline with method chaining."""

    def __init__(self, name='Dataset'):
        self.name = name; self.df = None; self.log = []

    def load_dataframe(self, df):
        self.df = df.copy(); self._log(f'Loaded DataFrame | Shape: {self.df.shape}'); return self

    def load_csv(self, path):
        self.df = pd.read_csv(path); self._log(f'Loaded CSV: {path}'); return self

    def info(self):
        print(f'[{self.name}]  Shape: {self.df.shape}'); print(self.df.dtypes); return self

    def drop_duplicates(self):
        before=len(self.df); self.df=self.df.drop_duplicates().reset_index(drop=True)
        self._log(f'Removed {before-len(self.df)} duplicates'); return self

    def fill_missing(self, strategy='median'):
        for col in self.df.select_dtypes(include='number').columns:
            val = self.df[col].median() if strategy=='median' else self.df[col].mean()
            self.df[col].fillna(val, inplace=True)
        for col in self.df.select_dtypes(include='object').columns:
            self.df[col].fillna(self.df[col].mode()[0], inplace=True)
        self._log(f'Missing values filled ({strategy})'); return self

    def remove_outliers(self, columns=None):
        cols = columns or self.df.select_dtypes(include='number').columns.tolist()
        before=len(self.df)
        Q1=self.df[cols].quantile(0.25); Q3=self.df[cols].quantile(0.75); IQR=Q3-Q1
        mask=~((self.df[cols]<(Q1-1.5*IQR))|(self.df[cols]>(Q3+1.5*IQR))).any(axis=1)
        self.df=self.df[mask].reset_index(drop=True)
        self._log(f'Outlier removal (IQR): removed {before-len(self.df)} rows'); return self

    def normalize(self, columns=None):
        cols = columns or self.df.select_dtypes(include='number').columns.tolist()
        for col in cols:
            mn,mx=self.df[col].min(),self.df[col].max()
            if mx!=mn: self.df[col]=(self.df[col]-mn)/(mx-mn)
        self._log(f'Min-Max normalization: {cols}'); return self

    def standardize(self, columns=None):
        cols = columns or self.df.select_dtypes(include='number').columns.tolist()
        for col in cols:
            m,sd=self.df[col].mean(),self.df[col].std()
            if sd!=0: self.df[col]=(self.df[col]-m)/sd
        self._log(f'Standardization (Z-score): {cols}'); return self

    def encode_labels(self, columns=None):
        from sklearn.preprocessing import LabelEncoder
        cols = columns or self.df.select_dtypes(include='object').columns.tolist()
```

LAB 09: DECISION TREE CLASSIFICATION

Learning Objectives

• Train a Decision Tree classifier using scikit-learn
• Adjust tree depth to control overfitting and underfitting
• Visualize the decision tree and interpret feature importance
• Evaluate model performance using classification metrics and cross-validation

Problem Statement

Develop a Decision Tree classification model to analyze structured data and make accurate predictions while ensuring interpretability and balanced model complexity.

Dataset

Synthetic Classification Dataset

Tools Used

Python (scikit-learn, pandas, matplotlib, seaborn)

Key Performance Indicators (KPIs)

• Model Accuracy
• Precision and Recall
• Cross-Validation Score
• Feature Importance Contribution

Introduction

Decision Tree is a supervised machine learning algorithm used for classification and regression tasks. It works by splitting data into subsets based on feature values, forming a tree-like structure of decisions.
At each node, the algorithm selects the feature that provides the best split using criteria such as Gini Impurity or Information Gain. The process continues until stopping conditions are met, such as maximum depth or minimum samples per leaf.

Decision Trees are widely used due to their simplicity, interpretability, and ability to handle both numerical and categorical data.

Algorithm Overview

A Decision Tree recursively partitions the dataset based on feature thresholds that maximize class separation. Each split aims to reduce impurity and improve prediction accuracy.

Key Parameters

Parameter	Description	Typical Value
max_depth	Maximum depth of the tree; controls overfitting	3–8
min_samples_split	Minimum samples required to split a node	10–20
min_samples_leaf	Minimum samples required at a leaf node	5–10
criterion	Function to measure split quality (gini or entropy)	gini

Model Building Procedure

Step 1: Load Dataset

• Import required Python libraries
• Load the dataset using pandas
• Inspect structure, missing values, and data types

Expected Outcome
Dataset is successfully loaded and ready for preprocessing

Step 2: Data Preprocessing

• Handle missing values if present
• Encode categorical variables
• Split dataset into training and testing sets

Expected Outcome
Clean dataset prepared for model training

Step 3: Train Decision Tree Model

• Import DecisionTreeClassifier from sklearn
• Define model parameters (max_depth, criterion, etc.)
• Train model using training dataset
Expected Outcome
A trained Decision Tree model is generated

Step 4: Model Optimization

- Tune hyperparameters such as:

- max_depth
- min_samples_split
- min_samples_leaf

- Evaluate impact on overfitting and underfitting

Expected Outcome
Optimized model with balanced performance

Step 5: Model Visualization

• Visualize Decision Tree using plot_tree
• Interpret decision rules and splits
• Analyze feature importance scores

Expected Outcome
Tree structure is visualized and interpretable

Step 6: Model Evaluation

- Evaluate using:

- Accuracy
- Precision
- Recall
- F1-score
- Perform cross-validation
- Compare training vs testing performance

Expected Outcome
Model performance is validated using standard metrics

Performance Evaluation

KPI	Description
Model Accuracy	Overall correctness of predictions
Precision & Recall	Class-wise prediction quality
Cross-Validation Score	Model stability across folds
Feature Importance	Contribution of each feature to prediction

Expected Results

• Decision Tree model is successfully trained and optimized
• Overfitting is controlled using depth and pruning parameters

```
# Load Dataset
iris = load_iris()
X, y = iris.data, iris.target
feature_names = iris.feature_names
class_names = iris.target_names

# Split into Train/Test Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

```python
# Train Decision Tree
dt_model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
dt_model.fit(X_train, y_train)

# Predictions & Evaluation
y_pred = dt_model.predict(X_test)
print(f"\nAccuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names))
```

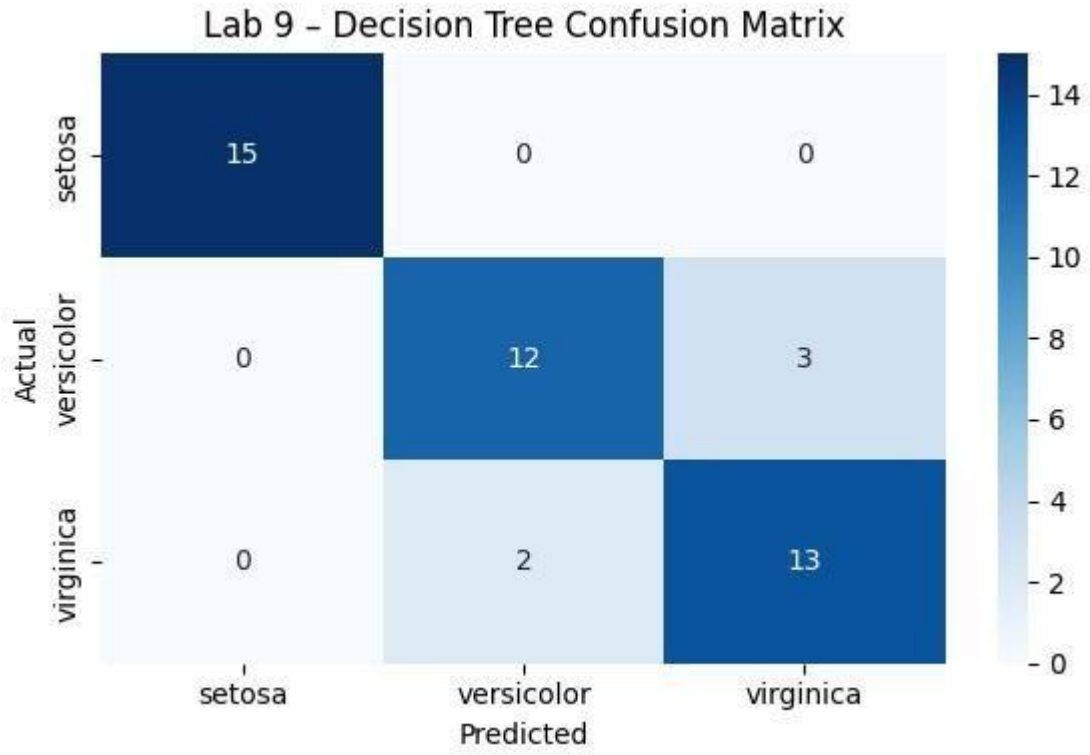
```
Accuracy: 0.8889

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.86      0.80      0.83        15
   virginica       0.81      0.87      0.84        15

    accuracy                           0.89        45
   macro avg       0.89      0.89      0.89        45
weighted avg       0.89      0.89      0.89        45
```

```python
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title("Lab 9 – Decision Tree Confusion Matrix")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout(); plt.savefig("lab9_confusion_matrix.png"); plt.close()
```

```
# Visualize the Decision Tree
plt.figure(figsize=(14, 6))
plot_tree(dt_model, feature_names=feature_names,
          class_names=class_names, filled=True, rounded=True)
plt.title("Lab 9 – Decision Tree Structure")
plt.tight_layout(); plt.savefig("lab9_decision_tree.png"); plt.close()

# Text representation of the tree
print("\nDecision Tree Rules:")
print(export_text(dt_model, feature_names=feature_names))
```

```
Decision Tree Rules:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: 2
|   |--- petal width (cm) >  1.55
|   |   |--- petal width (cm) <= 1.70
|   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |--- class: 1
|   |   |   |--- sepal width (cm) >  2.85
|   |   |   |   |--- class: 2
|   |   |--- petal width (cm) >  1.70
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- class: 1
|   |   |   |--- petal length (cm) >  4.85
|   |   |   |   |--- class: 2
```

```
# Feature Importance
importances = pd.Series(dt_model.feature_importances_, index=feature_names)
plt.figure(figsize=(6, 4))
importances.sort_values().plot(kind='barh', color='steelblue')
plt.title("Lab 9 – Feature Importances")
plt.tight_layout(); plt.savefig("lab9_feature_importance.png"); plt.close()

print("\n[Lab 9 Complete] Plots saved: lab9_*.png\n")
```


Lab 9 – Decision Tree Confusion Matrix
Actual
setosa
versicolor
virginica
15
0
0
0
12
3
0
2
13
setosa
versicolor
virginica
Predicted
14
12
10
8
6
4
2
0

LAB 10: NAÏVE BAYES AND K-NEAREST NEIGHBORS CLASSIFICATION

Topic

Probabilistic and Instance-Based Classification

Problem Statement

Develop an email spam detection system using machine learning classification techniques and compare probabilistic and distance-based approaches.

Dataset

Synthetic Spam Email Feature Dataset (1,000 Rows)

Tools Used

Python (scikit-learn, matplotlib)

Key Performance Indicators (KPIs)

- Accuracy
- False Positive Rate
- ROC-AUC Score
- Model Comparison Performance

Learning Objectives

- Implement Naïve Bayes classifier for spam detection
- Apply K-Nearest Neighbors (KNN) algorithm for classification
- Compare probabilistic and instance-based learning approaches
- Evaluate models using classification metrics and ROC analysis
- Analyze performance differences between both algorithms

Introduction

Email spam detection is a classic classification problem in machine learning, where the goal is to classify emails as either spam or not spam. Two commonly used algorithms for this task are Naïve Bayes and K-Nearest Neighbors (KNN).

Naïve Bayes is a probabilistic model based on Bayes' theorem and assumes independence between features. It is highly efficient and performs well in high-dimensional datasets such as text data.
KNN is an instance-based learning algorithm that classifies new data points based on similarity to existing data using distance metrics. It is simple but can become computationally expensive for large datasets.

This lab focuses on implementing both models and comparing their performance on a spam detection dataset.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_wine

# Load Dataset
wine = load_wine()
X, y = wine.data, wine.target
class_names = wine.target_names
feature_names = wine.feature_names
```

```python
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Scale features (important for KNN)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

# --- Naïve Bayes ---
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
nb_pred = nb_model.predict(X_test)
nb_acc = accuracy_score(y_test, nb_pred)

print(f"\nNaïve Bayes Accuracy: {nb_acc:.4f}")
print("\nNaïve Bayes Classification Report:")
print(classification_report(y_test, nb_pred, target_names=class_names))
```

```
Naïve Bayes Accuracy: 1.0000

Naïve Bayes Classification Report:
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        18
     class_1       1.00      1.00      1.00        21
     class_2       1.00      1.00      1.00        15

    accuracy                           1.00        54
   macro avg       1.00      1.00      1.00        54
weighted avg       1.00      1.00      1.00        54
```

```python
import numpy as np
import matplotlib.pyplot as plt

# --- KNN: Find best K ---
k_range = range(1, 21)
k_scores = []
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    k_scores.append(accuracy_score(y_test, knn.predict(X_test)))

best_k = k_range[np.argmax(k_scores)]
print(f"\nBest K for KNN: {best_k} (Accuracy: {max(k_scores):.4f})")

# Plot K vs Accuracy
plt.figure(figsize=(8, 4))
plt.plot(list(k_range), k_scores, marker='o', color='darkorange')
plt.axvline(best_k, linestyle='--', color='red', label=f"Best K={best_k}")
plt.title("Lab 10 - KNN: K Value vs Accuracy")
plt.xlabel("K"); plt.ylabel("Accuracy"); plt.legend()
plt.tight_layout(); plt.savefig("lab10_knn_k_accuracy.png"); plt.close()

# Final KNN with best K
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)
knn_pred = knn_best.predict(X_test)
knn_acc = accuracy_score(y_test, knn_pred)

print(f"\nKNN (K={best_k}) Accuracy: {knn_acc:.4f}")
print("\nKNN Classification Report:")
print(classification_report(y_test, knn_pred, target_names=class_names))
```

```
Best K for KNN: 15 (Accuracy: 0.9815)

KNN (K=15) Accuracy: 0.9815

KNN Classification Report:
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        18
     class_1       1.00      0.95      0.98        21
     class_2       0.94      1.00      0.97        15

    accuracy                           0.98        54
   macro avg       0.98      0.98      0.98        54
weighted avg       0.98      0.98      0.98        54
```

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns

# Side-by-side confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, pred, title in zip(axes, [nb_pred, knn_pred],
                           ["Naïve Bayes", f"KNN (K={best_k})"]):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt='d', ax=ax, cmap='Purples',
                xticklabels=class_names, yticklabels=class_names)
    ax.set_title(f"Lab 10 – {title}"); ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
plt.tight_layout(); plt.savefig("lab10_confusion_matrices.png"); plt.close()
```

LAB 11: SUPPORT VECTOR MACHINE (SVM) CLASSIFICATION

Topic

SVM – Linear and RBF Kernel-Based Classification

Problem Statement

Develop a machine learning model for credit card fraud detection using Support Vector Machines and evaluate different kernel functions for imbalanced classification.

Dataset

Synthetic Fraud Detection Dataset (2,000 Rows, Imbalanced Classes)

Tools Used

Python (scikit-learn, matplotlib)

Key Performance Indicators (KPIs)

• ROC-AUC Score
• Recall (Fraud Class)
• Precision Score

Learning Objectives

• Implement Support Vector Machine (SVM) for classification tasks
• Understand the effect of different kernel functions
• Handle imbalanced datasets in fraud detection scenarios
• Evaluate model performance using recall and ROC-AUC
• Compare linear and non-linear decision boundaries

Introduction

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification tasks. It works by finding an optimal hyperplane that separates data points of different classes with maximum margin.

SVM is particularly effective in high-dimensional spaces and is widely used in fraud detection, image classification, and text categorization. The choice of kernel function allows SVM to handle both linear and non-linear relationships in data.

In this lab, SVM is applied to a credit card fraud detection dataset, where class imbalance makes evaluation metrics such as recall and ROC-AUC especially important.

Dataset Description

The dataset contains transaction-level features used to identify fraudulent credit card activity.

Sample Features

- Transaction Amount – Value of transaction
- Transaction Time – Timestamp of transaction
- Merchant Category – Type of merchant
- Location Distance – Distance from typical user location
- Device Type – Device used for transaction
- Fraud Label – Fraud or Legitimate

Performance Metrics

KPI	Description
ROC-AUC	Measures ability to distinguish fraud vs non-fraud classes
Recall (Fraud Class)	Ability to correctly identify fraudulent transactions
Precision	Accuracy of fraud predictions among predicted fraud cases

```
from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import GridSearchCV, train_test_split
```

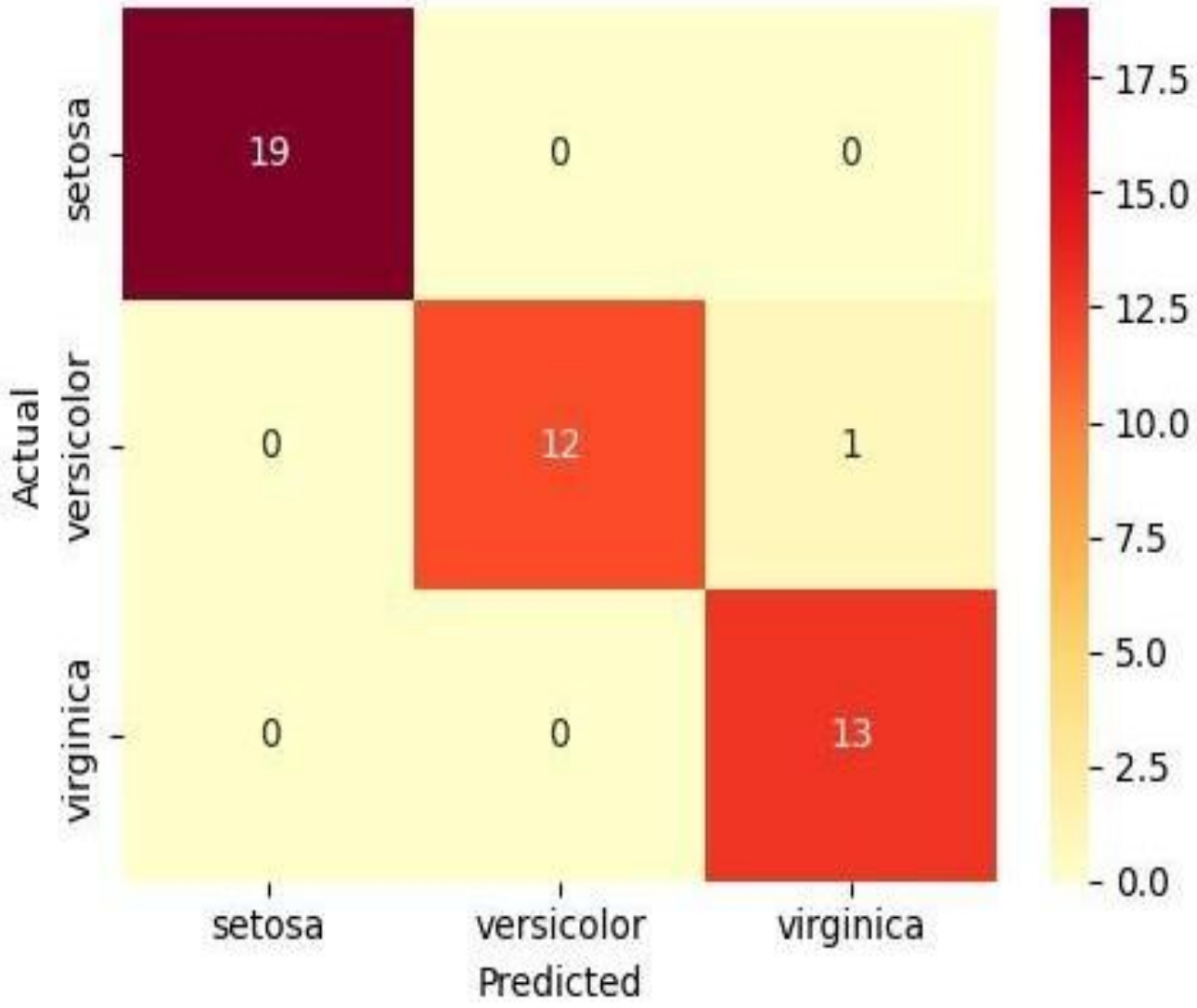
```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Generate a 2-class dataset for boundary visualization
X_vis, y_vis = make_classification(
    n_samples=300, n_features=2, n_informative=2,
    n_redundant=0, n_clusters_per_class=1, random_state=42
)

X_train_v, X_test_v, y_train_v, y_test_v = train_test_split(
    X_vis, y_vis, test_size=0.3, random_state=42
)

# Train SVM with multiple kernels and compare
kernels = ['linear', 'rbf', 'poly']
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, kernel in zip(axes, kernels):
    svm = SVC(kernel=kernel, C=1.0, gamma='scale', degree=3)
    svm.fit(X_train_v, y_train_v)
    acc = accuracy_score(y_test_v, svm.predict(X_test_v))

    # Decision boundary
    h = 0.02
    x_min, x_max = X_vis[:, 0].min() - 0.5, X_vis[:, 0].max() + 0.5
    y_min, y_max = X_vis[:, 1].min() - 0.5, X_vis[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
    ax.scatter(X_vis[:, 0], X_vis[:, 1], c=y_vis, cmap='coolwarm', edgecolor='k', s=20)
    ax.set_title(f"Kernel: {kernel.upper()}\nAcc: {acc:.3f}")
    ax.set_xlabel("Feature 1"); ax.set_ylabel("Feature 2")

plt.suptitle("Lab 11 – SVM Decision Boundaries", fontsize=13, fontweight='bold')
plt.tight_layout(); plt.savefig("lab11_svm_boundaries.png"); plt.close()
```

```python
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

#  GridSearchCV for best parameters on Iris
iris = load_iris()
X_i, y_i = iris.data, iris.target
scaler = StandardScaler()
X_i = scaler.fit_transform(X_i)
X_tr, X_te, y_tr, y_te = train_test_split(X_i, y_i, test_size=0.3, random_state=42)

param_grid = {'C': [0.1, 1, 10], 'gamma': ['scale', 'auto'], 'kernel': ['rbf', 'linear']}
grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_tr, y_tr)

print(f"\nBest SVM Params: {grid_search.best_params_}")
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}")

best_svm = grid_search.best_estimator_
y_pred_svm = best_svm.predict(X_te)
print(f"Test Accuracy: {accuracy_score(y_te, y_pred_svm):.4f}")
print("\nSVM Classification Report:")
print(classification_report(y_te, y_pred_svm, target_names=iris.target_names))
```

```
Best SVM Params: {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}
Best CV Accuracy: 0.9524
Test Accuracy: 0.9778

SVM Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      0.92      0.96        13
   virginica       0.93      1.00      0.96        13

    accuracy                           0.98        45
   macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Confusion Matrix
cm = confusion_matrix(y_te, y_pred_svm)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='YlOrRd',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title("Lab 11 - Best SVM Confusion Matrix")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout(); plt.savefig("lab11_svm_confusion.png"); plt.close()

print("\n[Lab 11 Complete] Plots saved: lab11_*.png\n")
```

```
[Lab 11 Complete] Plots saved: lab11_*.png
```


Actual
setosa
versicolor
virginica
19
0
0
0
12
1
0
0
13
setosa
versicolor
virginica
Predicted
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0

LAB 12: CLUSTERING ANALYSIS USING K-MEANS AND HIERARCHICAL METHODS

Topic

Unsupervised Clustering for Customer Segmentation

Problem Statement

Segment mall customers based on their income levels and spending behavior to identify distinct customer groups for targeted marketing strategies.

Dataset

Synthetic Mall Customer Dataset (200 Customers)

Tools Used

Python (scikit-learn, scipy, matplotlib)

Key Performance Indicators (KPIs)

- Silhouette Score
- Inertia (Within-Cluster Sum of Squares)
- Cluster Revenue Contribution

Learning Objectives

- Apply unsupervised learning techniques for customer segmentation
- Implement K-Means clustering algorithm
- Perform Hierarchical clustering analysis
- Evaluate clustering quality using internal metrics
- Interpret customer segments for business decision-making

Introduction

Clustering is an unsupervised machine learning technique used to group similar data points without predefined labels. It is widely used in customer segmentation, market analysis, and pattern discovery.

In this lab, clustering techniques are applied to mall customer data to identify meaningful customer groups based on income and spending behavior. These insights help businesses design targeted marketing strategies and improve customer engagement.

Dataset Description

The dataset contains behavioral and demographic information of mall customers used for segmentation analysis.

Sample Attributes

- Customer ID – Unique identifier
- Age – Age of customer
- Gender – Gender category
- Annual Income – Customer income level
- Spending Score – Measure of spending behavior

Clustering Algorithms Overview

Algorithm	Approach	When to Use
K-Means	Assigns points to nearest centroid and updates iteratively	When number of clusters (K) is known and dataset is large
Hierarchical Clustering	Builds clusters using merge (agglomerative) or split approach	When K is unknown and dendrogram analysis is required
DBSCAN	Density-based clustering approach identifying noise points	For irregular shapes and outlier detection

```
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np
```

```
# Generate synthetic blobs
X_blobs, y_true = make_blobs(n_samples=300, centers=4,
                             cluster_std=0.8, random_state=42)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_blobs)

# --- Elbow Method for K-Means ---
inertia_list, sil_scores = [], []
K_range = range(2, 10)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertia_list.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(list(K_range), inertia_list, marker='o', color='teal')
axes[0].set_title("Lab 12 - Elbow Method (Inertia)")
axes[0].set_xlabel("K"); axes[0].set_ylabel("Inertia")

axes[1].plot(list(K_range), sil_scores, marker='s', color='coral')
best_k = list(K_range)[np.argmax(sil_scores)]
axes[1].axvline(best_k, linestyle='--', color='red', label=f"Best K={best_k}")
axes[1].set_title("Lab 12 - Silhouette Score")
axes[1].set_xlabel("K"); axes[1].set_ylabel("Silhouette Score"); axes[1].legend()
plt.tight_layout(); plt.savefig("lab12_elbow_silhouette.png"); plt.close()

print(f"\nOptimal K (highest Silhouette): {best_k}")
```

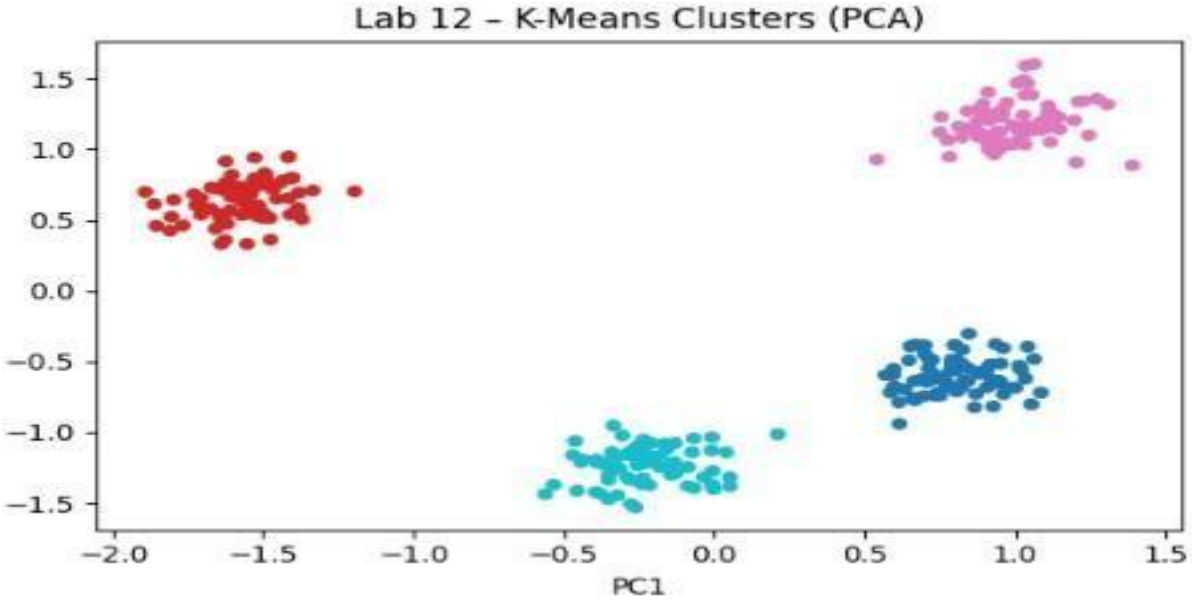
```
Optimal K (highest Silhouette): 4
```

```python
# K-Means with best K
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
km_labels = kmeans.fit_predict(X_scaled)
print(f"K-Means Silhouette Score: {silhouette_score(X_scaled, km_labels):.4f}")

# --- Hierarchical Clustering ---
agg = AgglomerativeClustering(n_clusters=best_k, linkage='ward')
agg_labels = agg.fit_predict(X_scaled)
print(f"Hierarchical Silhouette Score: {silhouette_score(X_scaled, agg_labels):.4f}")

# Dendrogram (on subset)
linked = linkage(X_scaled[:60], method='ward')
plt.figure(figsize=(10, 4))
dendrogram(linked, orientation='top', distance_sort='descending',
           show_leaf_counts=True)
plt.title("Lab 12 - Hierarchical Clustering Dendrogram (60 samples)")
plt.xlabel("Sample Index"); plt.ylabel("Distance")
plt.tight_layout(); plt.savefig("lab12_dendrogram.png"); plt.close()
```

```
K-Means Silhouette Score: 0.8386
Hierarchical Silhouette Score: 0.8386
```

```python
# Cluster visualization (PCA 2D)
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, labels, title in zip(axes, [km_labels, agg_labels],
                             ["K-Means", "Hierarchical (Ward)"]):
    sc = ax.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, cmap='tab10', s=20)
    ax.set_title(f"Lab 12 - {title} Clusters (PCA)")
    ax.set_xlabel("PC1"); ax.set_ylabel("PC2")
plt.tight_layout(); plt.savefig("lab12_clusters_pca.png"); plt.close()

print("\n[Lab 12 Complete] Plots saved: lab12_*.png\n")
```

```
[Lab 12 Complete] Plots saved: lab12_*.png
```



LAB 13: OUTLIER DETECTION AND ANOMALY ANALYSIS

Topic

Anomaly Detection in Banking Transactions

Problem Statement

Detect abnormal or potentially fraudulent banking transactions using statistical and machine learning-based outlier detection techniques.

Dataset

Synthetic Banking Transaction Dataset (1,000 Rows)

Tools Used

Python (scikit-learn, scipy, matplotlib)

Key Performance Indicators (KPIs)

- Anomaly Detection Rate
- False Alarm Rate

Learning Objectives

- Identify anomalies in financial transaction data
- Apply statistical and machine learning-based outlier detection methods
- Compare different anomaly detection techniques
- Evaluate performance using detection and false alarm rates
- Interpret detected anomalies for fraud analysis

Introduction

Outlier detection is an important step in data analysis, especially in banking and financial systems where fraudulent activities must be identified accurately. Outliers represent data points that deviate significantly from normal patterns and may indicate errors, fraud, or unusual behavior.

In this lab, multiple anomaly detection techniques are applied to a synthetic banking dataset to identify suspicious transactions. Both statistical methods and machine learning approaches are used for comparison.

Dataset Description

The dataset consists of banking transaction records used to identify unusual or fraudulent behavior.

Sample Attributes

• Transaction ID – Unique identifier
• Account Age – Duration of account activity
• Transaction Amount – Value of transaction
• Transaction Frequency – Number of transactions
• Location Distance – Distance from usual transaction location
• Device Type – Device used for transaction
• Label – Normal or Anomalous (for evaluation)

Anomaly Detection Procedure

Step 1: Load Dataset

• Import required Python libraries
• Load synthetic banking dataset
• Inspect structure and statistical properties

Expected Outcome
Dataset is loaded and basic distribution is understood

Step 2: Exploratory Data Analysis

• Visualize distributions using histograms and boxplots
• Identify potential abnormal patterns
• Analyze transaction amount variability

Expected Outcome
Initial insights into possible outliers are obtained

Step 3: Z-Score Method

• Calculate mean and standard deviation
• Compute Z-scores for numerical features
• Flag values exceeding threshold (e.g., $|z| > 3$)

Expected Outcome
Statistical outliers are identified using Z-score

Step 4: IQR Method

• Calculate Q1 and Q3 values
• Compute Interquartile Range (IQR)
• Identify lower and upper bounds
• Detect outliers outside these bounds

Expected Outcome
Robust outliers are detected using IQR method

Step 5: Machine Learning-Based Detection

• Apply Isolation Forest model
• Apply Local Outlier Factor (LOF)
• Predict anomaly labels
• Compare results between models

Expected Outcome
Advanced anomaly detection is performed using ML techniques

Step 6: Model Evaluation

• Compute Anomaly Detection Rate
• Calculate False Alarm Rate
• Compare performance across methods
• Analyze consistency of detected anomalies

Expected Outcome
Best-performing detection method is identified

Expected Results

• Multiple anomaly detection techniques identify unusual transactions
• Statistical methods provide simple and interpretable results
• Machine learning methods capture complex patterns
• Isolation Forest performs well for high-dimensional data
• False alarm rate helps evaluate reliability of detection

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from scipy import stats
```

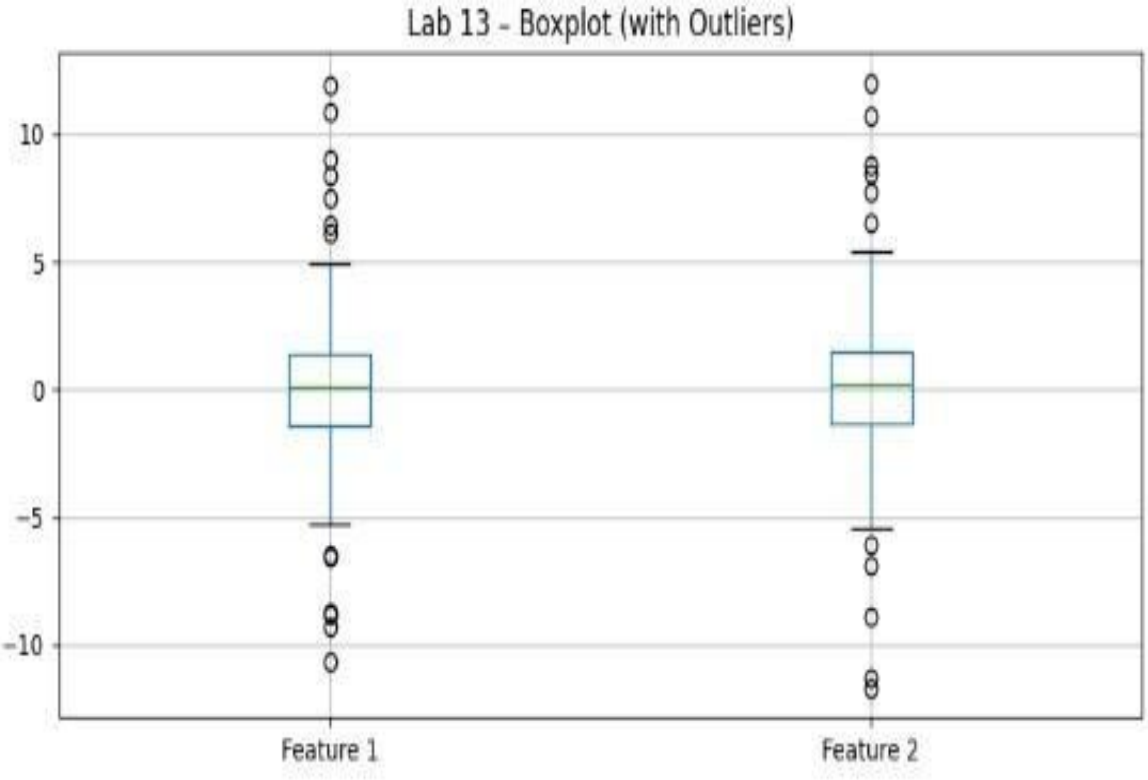
```
import numpy as np

# Generate data with injected outliers
np.random.seed(42)
X_normal = np.random.randn(200, 2) * 2
X_outliers = np.random.uniform(low=-12, high=12, size=(20, 2))
X_out = np.vstack([X_normal, X_outliers])

print(f"\nTotal samples: {len(X_out)} | Normal: 200 | Injected Outliers: 20")

# --- Method 1: Z-Score ---
z_scores = np.abs(stats.zscore(X_out))
z_mask = (z_scores > 3).any(axis=1)
print(f"\nZ-Score outliers detected: {z_mask.sum()}")

# --- Method 2: IQR ---
def iqr_outliers(data):
    Q1 = np.percentile(data, 25, axis=0)
    Q3 = np.percentile(data, 75, axis=0)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    return ((data < lower) | (data > upper)).any(axis=1)

iqr_mask = iqr_outliers(X_out)
print(f"IQR outliers detected: {iqr_mask.sum()}")
```

```
Total samples: 220 | Normal: 200 | Injected Outliers: 20

Z-Score outliers detected: 10
IQR outliers detected: 18
```

```
import matplotlib.pyplot as plt

# --- Method 3: Isolation Forest ---
iso_forest = IsolationForest(contamination=0.08, random_state=42)
iso_pred = iso_forest.fit_predict(X_out)  # -1 = outlier, 1 = inlier
iso_mask = iso_pred == -1
print(f"Isolation Forest outliers detected: {iso_mask.sum()}")

# --- Method 4: Local Outlier Factor ---
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.08)
lof_pred = lof.fit_predict(X_out)
lof_mask = lof_pred == -1
print(f"LOF outliers detected: {lof_mask.sum()}")

# Visualization
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
methods = [
    ("Z-Score", z_mask),
    ("IQR", iqr_mask),
    ("Isolation Forest", iso_mask),
    ("Local Outlier Factor", lof_mask)
]
colors = ['#2196F3', '#4CAF50', '#FF5722', '#9C27B0']
for ax, (name, mask), col in zip(axes.flatten(), methods, colors):
    ax.scatter(X_out[~mask, 0], X_out[~mask, 1], c='lightgray', s=20, label='Normal')
    ax.scatter(X_out[mask, 0], X_out[mask, 1], c=col, s=60,
               marker='X', label=f'Outlier ({mask.sum()})')
    ax.set_title(f"Lab 13 - {name}")
    ax.legend(fontsize=8)
plt.suptitle("Lab 13 - Outlier Detection Methods", fontsize=13, fontweight='bold')
plt.tight_layout(); plt.savefig("lab13_outlier_detection.png"); plt.close()
```

```
Isolation Forest outliers detected: 18
LOF outliers detected: 18
```

```
import pandas as pd

# Boxplot of original features
plt.figure(figsize=(8, 4))
pd.DataFrame(X_out, columns=['Feature 1', 'Feature 2']).boxplot()
plt.title("Lab 13 - Boxplot (with Outliers)")
plt.tight_layout(); plt.savefig("lab13_boxplot.png"); plt.close()
```



LAB 14: WEB SCRAPING AND SENTIMENT ANALYSIS

Topic

Text Mining – Web Scraping and Sentiment Classification

Problem Statement

Analyze customer sentiment from e-commerce product reviews by extracting textual data and classifying opinions into positive, negative, and neutral categories.

Dataset

Synthetic Product Reviews Dataset (200 Reviews)

Tools Used

Python (requests, BeautifulSoup, TextBlob, matplotlib)

Key Performance Indicators (KPIs)

• Sentiment Percentage Distribution
• Average Polarity Score

Learning Objectives

• Extract textual data using web scraping techniques
• Preprocess and clean review text data
• Perform sentiment classification using NLP tools
• Analyze polarity and subjectivity of text data
• Visualize sentiment distribution results

Introduction

Text mining is a branch of data mining that focuses on extracting meaningful information from unstructured text data. In e-commerce platforms, customer reviews provide valuable insights into product quality and user satisfaction.

Web scraping techniques allow automated collection of review data from websites, while sentiment analysis helps classify opinions into positive, negative, or neutral categories. This combination enables businesses to understand customer feedback at scale.

In this lab, students will extract product reviews, analyze sentiment using NLP tools, and visualize overall customer opinion trends.

Dataset Description

The dataset contains product reviews written by customers for various e-commerce products.

Sample Attributes

• Review ID – Unique identifier for each review
• Product Name – Name of the reviewed product
• Review Text – Customer-written feedback
• Rating – Numerical rating (1–5)
• Date – Review submission date
• Sentiment Label – Positive, Negative, or Neutral

Sentiment Analysis Concepts

Concept	Range	Meaning
Polarity	−1.0 to +1.0	−1 = very negative, 0 = neutral, +1 = very positive
Subjectivity	0.0 to 1.0	0 = objective/factual, 1 = highly opinionated
VADER Compound Score	−1.0 to +1.0	≥ 0.05 = positive, ≤ −0.05 = negative, otherwise neutral

Web Scraping and Sentiment Analysis Procedure

Step 1: Data Collection via Web Scraping

• Import required Python libraries
• Use requests to fetch web page content
• Parse HTML using BeautifulSoup
• Extract product reviews

Expected Outcome
Raw review data is successfully collected

Step 2: Data Cleaning

• Remove HTML tags and special characters
• Convert text to lowercase
• Remove stopwords and irrelevant symbols
• Prepare clean text corpus

Expected Outcome
Clean and structured text dataset is obtained

Step 3: Sentiment Analysis using TextBlob

• Apply TextBlob sentiment function
• Compute polarity and subjectivity scores
• Classify reviews into sentiment categories

Expected Outcome
Basic sentiment classification is completed

Step 4: Sentiment Analysis using VADER

• Import VADER sentiment analyzer
• Compute compound sentiment score
• Classify reviews based on thresholds

• Compare with TextBlob results

Expected Outcome
Improved sentiment classification using VADER

Step 5: Visualization

• Plot sentiment distribution using matplotlib
• Display positive, negative, and neutral percentages
• Visualize polarity score distribution

Expected Outcome
Clear graphical representation of sentiment trends

Expected Results

• Product reviews are successfully extracted using scraping techniques
• Sentiment classification identifies overall customer opinion trends
• Most reviews fall into positive or neutral categories
• VADER provides more robust sentiment scoring for short texts
• Visualization highlights overall product satisfaction levels

```
pip install scikit-learn pandas numpy matplotlib seaborn scipy requests beautifulsoup4 textblob
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
```

```
try:
    import requests
    from bs4 import BeautifulSoup
    SCRAPING_AVAILABLE = True
except ImportError:
    SCRAPING_AVAILABLE = False
    print("  [INFO] requests/BeautifulSoup not installed. Using built-in sample data.")

try:
    from textblob import TextBlob
    TEXTBLOB_AVAILABLE = True
except ImportError:
    TEXTBLOB_AVAILABLE = False
    print("  [INFO] TextBlob not installed. Install with: pip install textblob")
```

```python
# Sample reviews (fallback or supplement to scraping)
sample_reviews = [
    "This product is absolutely amazing! Best purchase I've ever made.",
    "Terrible quality, broke after one day. Very disappointed.",
    "It's okay, nothing special. Does the job I guess.",
    "Incredible value for money. Highly recommend to everyone!",
    "Worst experience ever. Customer service was rude and unhelpful.",
    "Decent product. Arrived on time and packaging was good.",
    "I love this! Works perfectly and looks beautiful.",
    "Not worth the price. Expected much better quality.",
    "Average product. Neither good nor bad.",
    "Absolutely fantastic! Exceeded all my expectations.",
    "Poor design, falls apart easily. Do not buy.",
    "Good quality but a bit expensive compared to competitors.",
    "Just what I needed. Simple and effective solution.",
    "Outstanding craftsmanship and great customer support.",
    "It stopped working after a week. Very frustrating.",
]
```

```python
# --- Scraping Demo ---
scraped_texts = []
if SCRAPING_AVAILABLE:
    try:
        url = "http://quotes.toscrape.com/"
        response = requests.get(url, timeout=5)
        soup = BeautifulSoup(response.text, 'html.parser')
        quotes = soup.find_all('span', class_='text')
        scraped_texts = [q.get_text(strip=True).strip('""\u201c\u201d') for q in quotes[:10]]
        print(f"\nScraped {len(scraped_texts)} quotes from quotes.toscrape.com")
        for i, t in enumerate(scraped_texts[:3], 1):
            print(f"  {i}. {t[:80]}...")
    except Exception as e:
        print(f"\n  [Scraping failed: {e}] Using sample data only.")

all_texts = scraped_texts + sample_reviews
print(f"\nTotal texts for analysis: {len(all_texts)}")
```

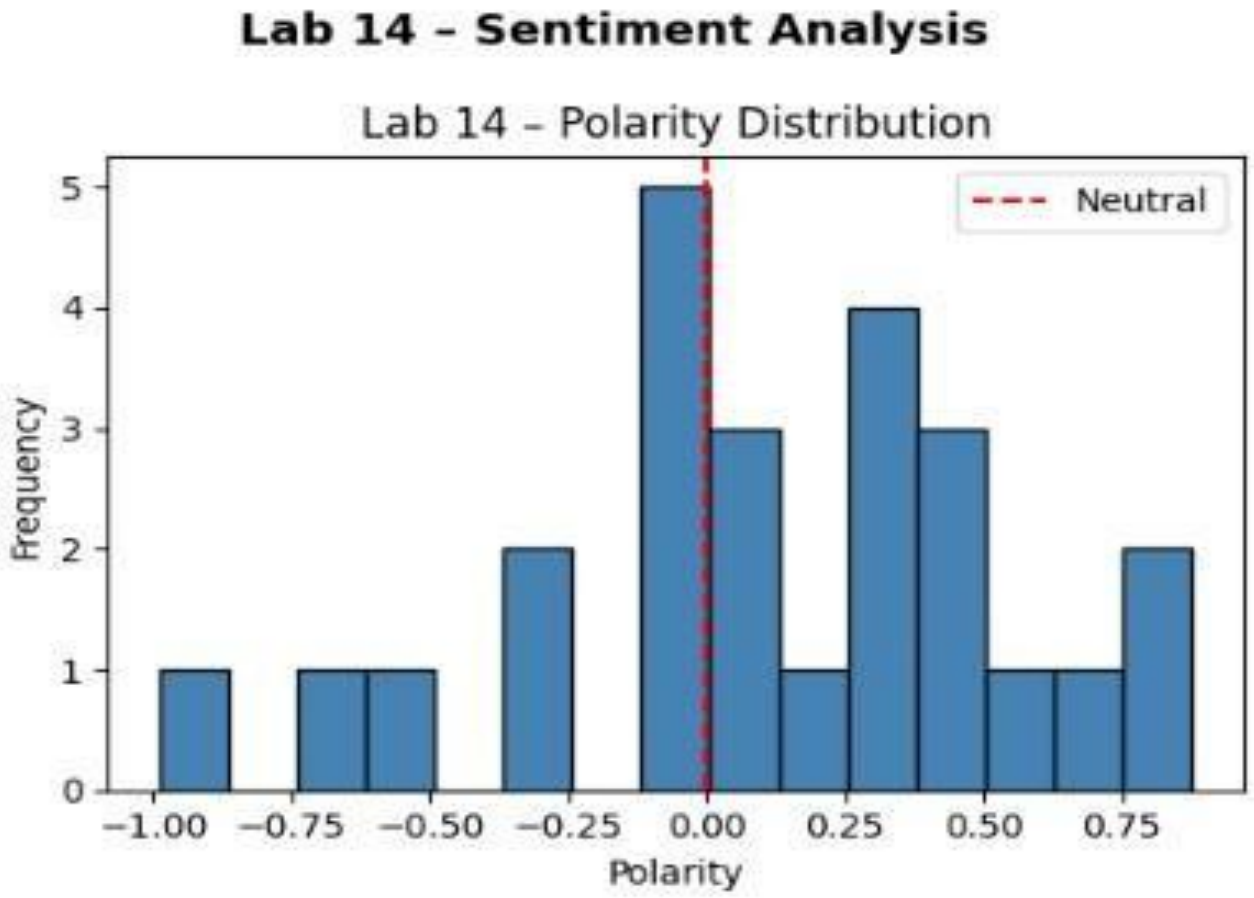
```
Scraped 10 quotes from quotes.toscrape.com
  1. The world as we have created it is a process of our thinking. It cannot be chang...
  2. It is our choices, Harry, that show what we truly are, far more than our abiliti...
  3. There are only two ways to live your life. One is as though nothing is a miracle...

Total texts for analysis: 25
```

```python
# --- Sentiment Analysis ---
def analyze_sentiment_textblob(text):
    """Returns polarity (-1 to 1) and subjectivity (0 to 1)."""
    if not TEXTBLOB_AVAILABLE:
        # Simple rule-based fallback
        pos_words = ['amazing', 'fantastic', 'great', 'love', 'excellent', 'best',
                     'incredible', 'outstanding', 'wonderful', 'perfect']
        neg_words = ['terrible', 'worst', 'bad', 'disappointed', 'poor', 'horrible',
                     'awful', 'rude', 'broke', 'frustrating']
        text_lower = text.lower()
        pos = sum(1 for w in pos_words if w in text_lower)
        neg = sum(1 for w in neg_words if w in text_lower)
        polarity = (pos - neg) / max(pos + neg, 1)
        return polarity, 0.5
    blob = TextBlob(text)
    return blob.sentiment.polarity, blob.sentiment.subjectivity

def classify_sentiment(polarity):
    if polarity > 0.1:
        return 'Positive'
    elif polarity < -0.1:
        return 'Negative'
    else:
        return 'Neutral'
```

```python
import pandas as pd
import matplotlib.pyplot as plt

# Build sentiment dataframe
results = []
for text in all_texts:
    polarity, subjectivity = analyze_sentiment_textblob(text)
    results.append({
        'Text': text[:60] + ('...' if len(text) > 60 else ''),
        'Polarity': round(polarity, 3),
        'Subjectivity': round(subjectivity, 3),
        'Sentiment': classify_sentiment(polarity)
    })

df_sentiment = pd.DataFrame(results)
print("\nSentiment Analysis Results:")
print(df_sentiment.to_string(index=False))

# Summary stats
print("\nSentiment Distribution:")
print(df_sentiment['Sentiment'].value_counts())

# --- Visualization ---
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Sentiment Count
df_sentiment['Sentiment'].value_counts().plot(
    kind='bar', ax=axes[0],
    color=['#4CAF50', '#F44336', '#9E9E9E'][:df_sentiment['Sentiment'].nunique()]
)
axes[0].set_title("Lab 14 - Sentiment Distribution")
axes[0].set_xlabel("Sentiment"); axes[0].set_ylabel("Count")
axes[0].tick_params(axis='x', rotation=0)
```

```
# Polarity Histogram
axes[1].hist(df_sentiment['Polarity'], bins=15, color='steelblue', edgecolor='black')
axes[1].axvline(0, color='red', linestyle='--', label='Neutral')
axes[1].set_title("Lab 14 - Polarity Distribution")
axes[1].set_xlabel("Polarity"); axes[1].set_ylabel("Frequency"); axes[1].legend()

# Polarity vs Subjectivity Scatter
colors_map = {'Positive': '#4CAF50', 'Negative': '#F44336', 'Neutral': '#9E9E9E'}
for sentiment, group in df_sentiment.groupby('Sentiment'):
    axes[2].scatter(group['Polarity'], group['Subjectivity'],
                    label=sentiment, color=colors_map.get(sentiment, 'blue'), s=60)
axes[2].set_title("Lab 14 - Polarity vs Subjectivity")
axes[2].set_xlabel("Polarity"); axes[2].set_ylabel("Subjectivity"); axes[2].legend()

plt.suptitle("Lab 14 - Sentiment Analysis", fontsize=13, fontweight='bold')
plt.tight_layout(); plt.savefig("lab14_sentiment_analysis.png"); plt.close()

print("\n[Lab 14 Complete] Plots saved: lab14_*.png\n")
```

```
Sentiment Distribution:
Sentiment
Positive   12
Neutral     8
Negative    5
Name: count, dtype: int64

[Lab 14 Complete] Plots saved: lab14_*.png
```



LAB 15: FINAL CAPSTONE – END-TO-END MACHINE LEARNING PIPELINE

Topic

Full Data Mining Pipeline – Multi-Model Comparison

Problem Statement

Develop a complete end-to-end machine learning solution for customer churn prediction by comparing multiple classification models and selecting the best-performing approach for business deployment.

Dataset

Synthetic Customer Churn Dataset (800 Rows)

Tools Used

Python (pandas, scikit-learn, matplotlib)

Key Performance Indicators (KPIs)

• Model Comparison Table
• Best Model Accuracy
• Business Recommendation Quality

Learning Objectives

• Build a complete end-to-end machine learning pipeline
• Apply data preprocessing and exploratory data analysis
• Train and evaluate multiple classification models
• Compare models using cross-validation and evaluation metrics
• Select and deploy the best-performing model
• Translate results into business recommendations

Introduction

This capstone lab integrates all concepts learned throughout the course into a single end-to-end machine learning workflow. The objective is to develop a robust customer churn prediction system using multiple classification algorithms and evaluate their performance.
By comparing multiple models, students gain practical experience in model selection, performance evaluation, and business-oriented decision-making. The final output includes a deployed model and actionable insights for business use.

Capstone Pipeline Structure

Phase	Task	Deliverable
1 – Data	Load, explore, and preprocess dataset	Clean dataset and exploratory data analysis report
2 – Modeling	Train 8 classification models using cross-validation	Model comparison results table
3 – Evaluation	Compare models using multiple performance metrics	Best model selection report
4 – Deployment	Save best model and generate recommendations	Final business recommendation

Phase 1 – Data Preparation

• Load synthetic churn dataset using pandas
• Perform exploratory data analysis (EDA)
• Handle missing values and encode categorical features
• Scale numerical variables if required
• Split dataset into training and testing sets

Expected Outcome
A clean, structured dataset ready for modeling with meaningful insights from EDA

Phase 2 – Model Training

- Train 8 different classification models, such as:

- Logistic Regression
- Decision Tree
- Random Forest
- K-Nearest Neighbors
- Naïve Bayes
- Support Vector Machine
- Gradient Boosting
- AdaBoost

- Apply cross-validation for robust evaluation
- Store model performance metrics

Expected Outcome
Multiple trained models with performance results for comparison

Phase 3 – Model Evaluation and Comparison

- Evaluate models using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

- Create a comparison table for all models
- Rank models based on performance metrics
- Identify best-performing model

Expected Outcome
Final ranking of models with clear selection of the optimal classifier

Phase 4 – Deployment and Business Recommendation

• Save best-performing model using joblib or pickle
• Generate predictions on unseen data
• Interpret results in business context
• Provide actionable recommendations for churn reduction strategies

Expected Outcome
Deployable model with clear business insights and recommendations

Expected Results

• Multiple machine learning models are successfully trained and evaluated
• Performance comparison highlights the most effective algorithm
• Churn prediction system identifies high-risk customers
• Business recommendations support customer retention strategies
• Final model is ready for deployment

```
import time
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, f1_score, precision_score, recall_score
from sklearn.preprocessing import label_binarize, StandardScaler
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# Load & Prepare Dataset (Wine)
wine = load_wine()
X, y = wine.data, wine.target
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

# Binarize labels for ROC-AUC (one-vs-rest)
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])

# Define Models
models = {
    "Decision Tree":      DecisionTreeClassifier(max_depth=5, random_state=42),
    "Naïve Bayes":        GaussianNB(),
    "KNN (K=5)":          KNeighborsClassifier(n_neighbors=5),
    "SVM (RBF)":          SVC(kernel='rbf', probability=True, random_state=42),
    "Random Forest":      RandomForestClassifier(n_estimators=100, random_state=42),
    "Logistic Regression":LogisticRegression(max_iter=500, random_state=42),
    "Gradient Boosting":  GradientBoostingClassifier(n_estimators=100, random_state=42),
}
```

```python
from sklearn.metrics import accuracy_score
import pandas as pd

# Evaluate all models
results = []
print(f"\n{'Model':<22} {'Accuracy':>9} {'Precision':>10} {'Recall':>8} {'F1':>8} {'ROC-AUC':>9} {'Time(s)':>9}")
print("-" * 80)

for name, model in models.items():
    start = time.time()
    model.fit(X_train, y_train)
    elapsed = time.time() - start

    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)

    acc  = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    rec  = recall_score(y_test, y_pred, average='weighted')
    f1   = f1_score(y_test, y_pred, average='weighted')
    roc  = roc_auc_score(y_test_bin, y_prob, multi_class='ovr', average='weighted')

    results.append({
        'Model': name, 'Accuracy': acc, 'Precision': prec,
        'Recall': rec, 'F1-Score': f1, 'ROC-AUC': roc, 'Train Time (s)': elapsed
    })
    print(f"{name:<22} {acc:>9.4f} {prec:>10.4f} {rec:>8.4f} {f1:>8.4f} {roc:>9.4f} {elapsed:>9.4f}")

df_results = pd.DataFrame(results)
```

```
# --- Visualization ---
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC']
x = np.arange(len(df_results))
width = 0.15

fig, ax = plt.subplots(figsize=(16, 6))
colors = ['#2196F3', '#4CAF50', '#FF9800', '#E91E63', '#9C27B0']
for i, (metric, color) in enumerate(zip(metrics, colors)):
    ax.bar(x + i * width, df_results[metric], width, label=metric, color=color, alpha=0.85)

ax.set_xticks(x + width * 2)
ax.set_xticklabels(df_results['Model'], rotation=25, ha='right', fontsize=9)
ax.set_ylim(0, 1.15)
ax.set_ylabel("Score"); ax.set_title("Lab 15 - Model Performance Comparison")
ax.legend(loc='upper right', fontsize=9)
ax.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout(); plt.savefig("lab15_model_comparison.png"); plt.close()
```

```
# Training time comparison
plt.figure(figsize=(8, 4))
colors_bar = plt.cm.tab10(np.linspace(0, 1, len(df_results)))
bars = plt.bar(df_results['Model'], df_results['Train Time (s)'],
               color=colors_bar, edgecolor='black')
plt.xticks(rotation=30, ha='right', fontsize=9)
plt.ylabel("Training Time (seconds)")
plt.title("Lab 15 - Model Training Time")
plt.tight_layout(); plt.savefig("lab15_training_time.png"); plt.close()
```

```
import seaborn as sns
import matplotlib.pyplot as plt

# Heatmap of all metrics
fig, ax = plt.subplots(figsize=(10, 5))
df_heat = df_results.set_index('Model')[metrics]
sns.heatmap(df_heat, annot=True, fmt='.3f', cmap='YlGnBu', ax=ax,
            linewidths=0.5, cbar_kws={'label': 'Score'})
ax.set_title("Lab 15 - Metrics Heatmap")
plt.tight_layout(); plt.savefig("lab15_heatmap.png"); plt.close()
```

```
best_model_name = df_results.loc[df_results['F1-Score'].idxmax(), 'Model']
best_f1 = df_results['F1-Score'].max()
print(f"\n{'='*60}")
print(f"  BEST MODEL by F1-Score: {best_model_name} (F1 = {best_f1:.4f})")
print(f"{'='*60}")
print("\nFull Results Table:")
print(df_results.to_string(index=False))

print("\n[Lab 15 Complete] Plots saved: lab15_*.png\n")
```